

Lecture6a

October 5, 2025

1 CS5489- Machine Learning

2 Lecture 6a - Convolutional Neural Networks (CNNs)

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

- Convolutional neural network (CNN)
- Regularization

```
[1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
from scipy import signal
from prettytable import PrettyTable

rbow = plt.get_cmap('rainbow')
```

```
[2]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import struct
import sys
print(f"Python: {sys.version} PyTorch: {torch.__version__}")

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Python: 3.13.7 | packaged by Anaconda, Inc. | (main, Sep 9 2025, 19:54:17)
[Clang 17.0.6] PyTorch: 2.8.0
Using device: cpu

```
[3]: def plot_history(history):
    fig, ax1 = plt.subplots()

    ax1.plot(history['train_loss'], 'r', label="training loss ({:.6f})".
    ↪format(history['train_loss'][-1]))
    ax1.plot(history['val_loss'], 'r--', label="validation loss ({:.6f})".
    ↪format(history['val_loss'][-1]))
    ax1.grid(True)
    ax1.set_xlabel('iteration')
    ax1.legend(loc="best", fontsize=9)
    ax1.set_ylabel('loss', color='r')
    ax1.tick_params('y', colors='r')

    if 'train_accuracy' in history:
        ax2 = ax1.twinx()

        ax2.plot(history['train_accuracy'], 'b', label="training acc ({:.4f})".
        ↪format(history['train_accuracy'][-1]))
        ax2.plot(history['val_accuracy'], 'b--', label="validation acc ({:.
        ↪4f})".format(history['val_accuracy'][-1]))

        ax2.legend(loc="best", fontsize=9)
        ax2.set_ylabel('acc', color='b')
        ax2.tick_params('y', colors='b')
```

```
[4]: def show_imgs(W_list, nc=10, highlight_green=None, highlight_red=None,
    ↪titles=None):
    nfilter = len(W_list)
    nr = (nfilter - 1) // nc + 1
    for i in range(nr):
        for j in range(nc):
            idx = i * nc + j
            if idx == nfilter:
                break
            plt.subplot(nr, nc, idx + 1)
            cur_W = W_list[idx]
            plt.imshow(cur_W, cmap='gray', interpolation='nearest')
            if titles is not None:
                plt.title(titles % idx)

            if ((highlight_green is not None) and highlight_green[idx]) or \
                ((highlight_red is not None) and highlight_red[idx]):
                ax = plt.gca()
```

```

        if highlight_green[idx]:
            mycol = '#00FF00'
        else:
            mycol = 'r'
        for S in ['bottom', 'top', 'right', 'left']:
            ax.spines[S].set_color(mycol)
            ax.spines[S].set_lw(2.0)
        ax.xaxis.set_ticks_position('none')
        ax.yaxis.set_ticks_position('none')
        ax.set_xticks([])
        ax.set_yticks([])
    else:
        plt.gca().set_axis_off()

```

```

[5]: def read_32int(f):
        return struct.unpack('>I', f.read(4))[0]
def read_img(img_path):
    with open(img_path, 'rb') as f:
        magic_num = read_32int(f)
        num_image = read_32int(f)
        n_row = read_32int(f)
        n_col = read_32int(f)
        #print 'num_image = {}; n_row = {}; n_col = {}'.format(num_image,
↪n_row, n_col)
        res = []
        npixel = n_row * n_col
        res_arr = fromfile(f, dtype='B')
        res_arr = res_arr.reshape((num_image, n_row, n_col), order='C')
        #print 'image data shape = {}'.format(res_arr.shape)
        return num_image, n_row, n_col, res_arr
def read_label(label_path):
    with open(label_path, 'rb') as f:
        magic_num = read_32int(f)
        num_label = read_32int(f)
        #print 'num_label = {}'.format(num_label)
        res_arr = fromfile(f, dtype='B')
        #print res_arr.shape
        #res_arr = res_arr.reshape((num_label, 1))
        res_arr = res_arr.ravel()
        #print 'label data shape = {}'.format(res_arr.shape)
        return num_label, res_arr

```

```

[6]: n_train, nrow, ncol, training = read_img('data/train-images.idx3-ubyte')
    _, trainY = read_label('data/train-labels.idx1-ubyte')
    n_test, _, _, testing = read_img('data/t10k-images.idx3-ubyte')
    _, testY = read_label('data/t10k-labels.idx1-ubyte')

```

```
# for demonstration we only use 10% of the training data
sample_index = range(0, training.shape[0], 10)
training = training[sample_index]
trainY = trainY[sample_index]
print(training.shape)
print(trainY.shape)
print(testing.shape)
print(testY.shape)
```

```
(6000, 28, 28)
(6000,)
(10000, 28, 28)
(10000,)
```

4 Signals

- So far we have assumed the input $\mathbf{x}$ is a vector
 - or have turned 2D images into vectors.
- *What if the input has more structure?*
- For example:
 - **1-D signal** (time)

mono audio (1 feature)

stereo audio (2 features)

<tr><td></td><td></td></tr>

- **2-D signal** (space)
 - grayscale image (1 feature)
 - color image (3 features)
 - hyperspectral image (300 features)
- **3-D signal** (space+time, volume)

color video (3 features)

3D CT scan (1 feature)

<tr><td></td><td></td></tr>

5 Assumed Properties of Signals

- *Locality*
 - at low-level, features from 1 region are independent (do not depend on) features from a far-away region.
- *Translation*
 - the same features can appear anywhere in the signal.

6 Using the standard MLP layer...

- Each feature z_i is computed from all the inputs, but we only want local features (locality).
- The pattern could appear anywhere, but weights are trained for each location z_i separately (translation).
- For image input, we transform the image into a vector, which is the input into the MLP.
- **Problem:** This ignores the spatial relationship between pixels in the image.
 - Images contain local structures
 - * groups of neighboring pixels correspond to visual structures (edges, corners, texture).
 - * pixels far from each other are typically not correlated.
- **How to model the local structure of the signal?**
 - local features, feature translation
- **Answer:** Use a local feature extractor in the signal.

7 Convolution

- Consider 1D signal in *discrete* time: $x(t)$, $t \in \mathbb{Z}$
- Define the filter $w(t)$
 - “flipped” filter: $\tilde{w}(t) = w(-t)$

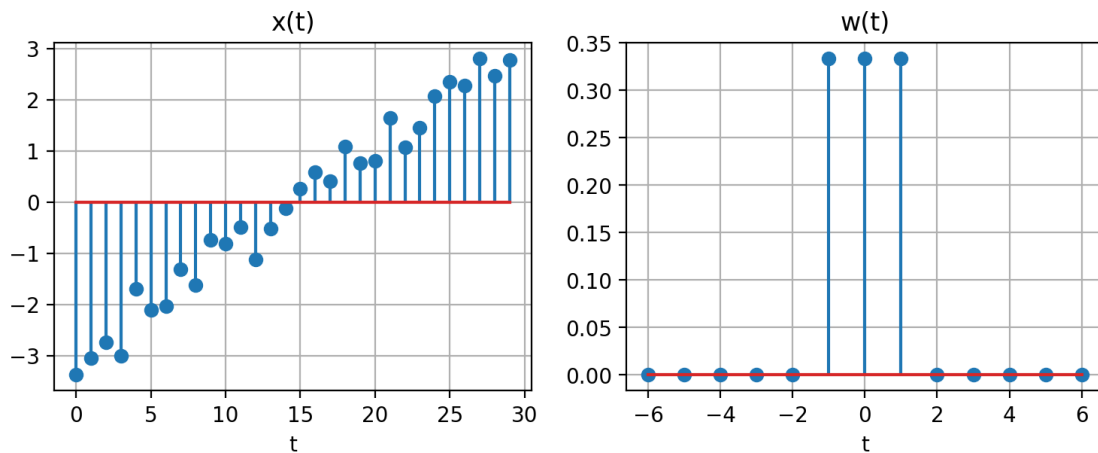
```
[7]: def zeropad(w, left0, right0):  
    return concatenate([zeros(left0,), w, zeros(right0,)])  
  
def zeropad2(w, left0, right0, up0, down0):  
    ws = w.shape;  
    x = concatenate([  
        zeros(shape=(up0, left0+ws[1]+right0)),  
        concatenate([  
            zeros(shape=(ws[0], left0)),  
            w,  
            zeros(shape=(ws[0], right0))  
        ], axis=1),  
        zeros(shape=(down0, left0+ws[1]+right0))  
    ])  
    return x
```

```
[8]: random.seed(4487)  
x = linspace(-3,3,30) + 0.3*random.normal(size=(30,))  
w = [1/3, 1/3, 1/3]  
ww = zeropad(w,5,5)  
tw = flip(w)  
  
xfig = plt.figure(figsize=(9,3))  
plt.subplot(1,2,1)  
plt.stem(x)
```

```
plt.grid(True)
plt.xlabel('t')
plt.title('x(t)')
plt.subplot(1,2,2)
plt.stem(arange(-6,7), ww)
plt.xlabel('t')
plt.title('w(t)')
plt.grid(True)
plt.close()
```

[9]: xfig

[9]:



- Convolution is a filtering operation
 - $s(t) = x * w = \sum_a x(a)w(t-a)$
 - * “*” is the symbol for convolution
- It's related to cross-correlation with the “flipped” filter.
 - $s(t) = \sum_a x(a)\tilde{w}(a-t)$
 - for a given t :
 1. shift $\tilde{w}$ by t
 2. multiply shifted $\tilde{w}$ with x
 3. sum to get $s(t)$

```
[10]: s = convolve(x, w)

sfig = plt.figure(figsize=(8,5))
plt.subplot(3,1,1)
plt.stem(x)
plt.xlim(-3,32); plt.grid(True)
plt.ylabel('$x(a)$')
plt.xlabel('a')
plt.subplot(3,1,2)
```

```

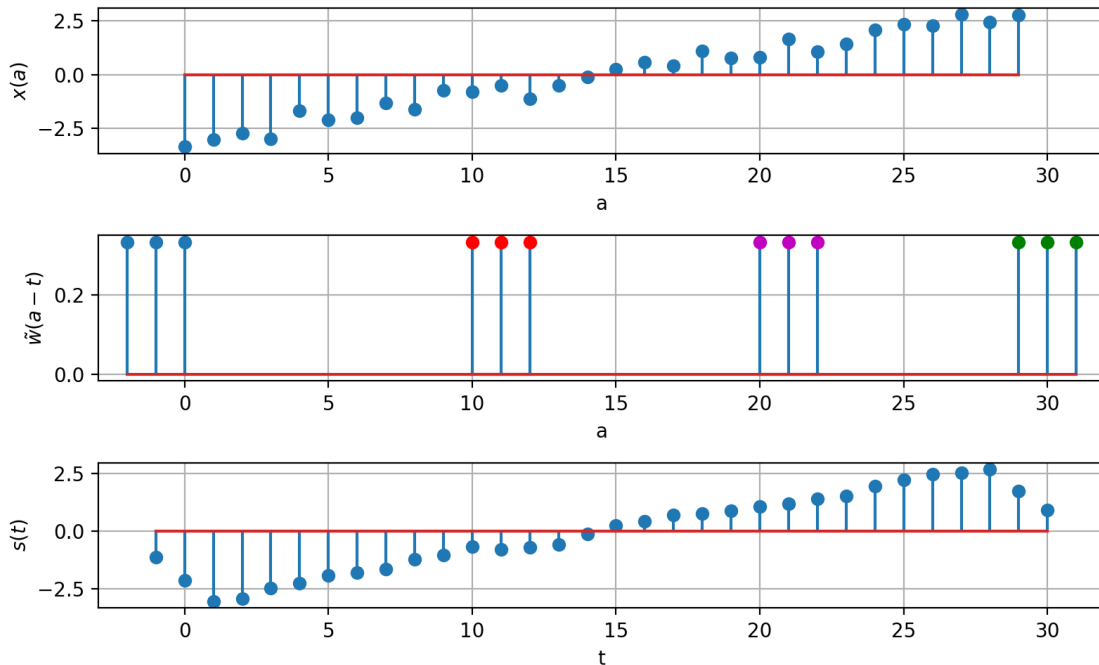
plt.stem(range(-2,1), tw)
plt.stem(range(10,13), tw, markerfmt='or')
plt.stem(range(20,23), tw, markerfmt='om')
plt.stem(range(29,32), tw, markerfmt='og')
plt.xlim(-3,32); plt.grid(True)
plt.ylabel('$\\tilde{w}(a-t)$')
plt.xlabel('a')
plt.stem(zeros(30,), markerfmt=',r')
plt.subplot(3,1,3)
plt.stem(range(-1,31), s)
plt.xlim(-3,32); plt.grid(True)
plt.ylabel('$s(t)$')
plt.xlabel('t')
#plt.subplots_adjust(hspace=0.35)
plt.tight_layout()
plt.close()

```

- $s(t) = \sum_a x(a) \tilde{w}(a-t)$
 1. shift $\tilde{w}$ by t
 2. multiply shifted $\tilde{w}$ with x
 3. sum to get $s(t)$

[11]: sfig

[11]:



- Boundary conditions

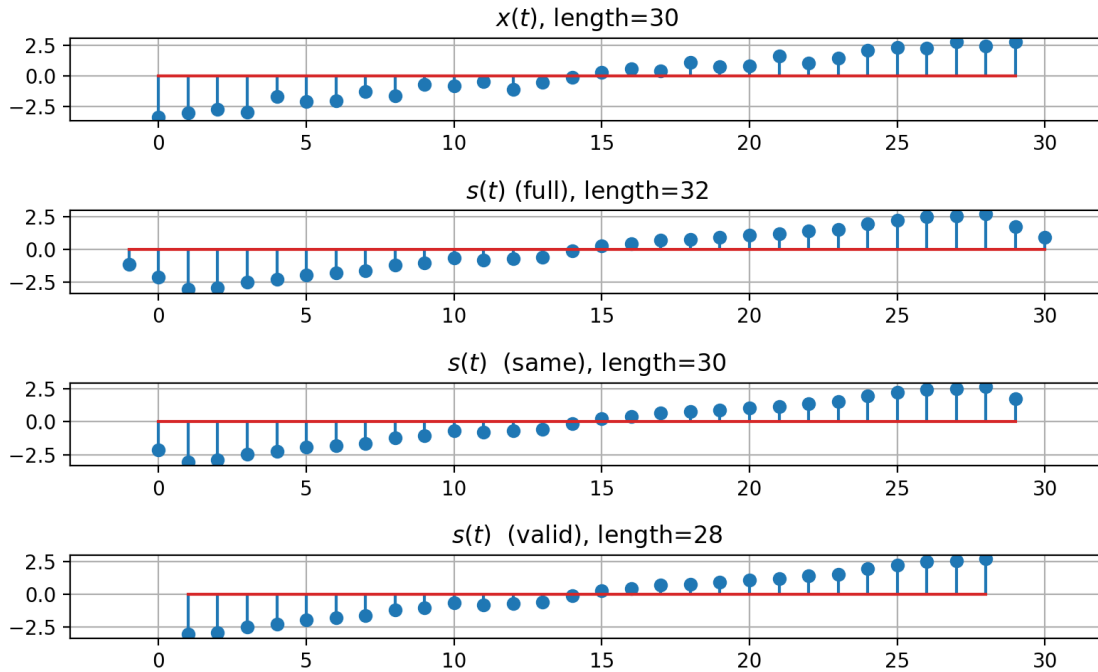
- It is assumed that the rest of the signal is all 0.
- The convolution result near the ends of the signal uses these “artificial” zeros.
 - the filtered signal is longer than the original signal
 - length increases by $P - 1$, where P is the non-zero extent of the filter.
- Three ways to handle this:
 - 1) use everything with *non-zero* response (‘full’ mode)
 - 2) keep the response the *same* length as the signal (‘same’ mode)
 - 3) keep only responses where the *entire* filter is on the signal (‘valid’ mode)

```
[12]: cfig = plt.figure(figsize=(8,5))
plt.subplot(4,1,1)
plt.stem(x)
plt.xlim(-3,32); plt.grid(True)
plt.title('$x(t)$, length=30')
plt.subplot(4,1,2)
plt.stem(range(-1,31), convolve(x,w,mode='full'))
plt.xlim(-3,32); plt.grid(True)
plt.title('$s(t)$ (full), length=32')
plt.subplot(4,1,3)
plt.stem(range(0,30), convolve(x,w,mode='same'))
plt.xlim(-3,32); plt.grid(True)
plt.title('$s(t)$ (same), length=30')
plt.subplot(4,1,4)
plt.stem(range(1,29), convolve(x,w,mode='valid'))
plt.xlim(-3,32); plt.grid(True)
plt.title('$s(t)$ (valid), length=28')

#plt.subplots_adjust(hspace=0.50)
plt.tight_layout()
plt.close()
```

```
[13]: cfig
```

```
[13]:
```

8 2D Convolution

- Straightforward to extend to multiple dimensions
- 2D discrete convolution

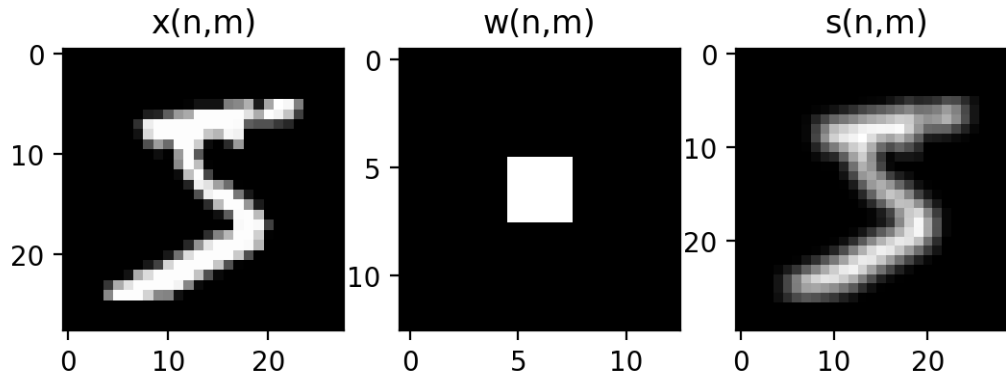
$$s(n, m) = x * w = \sum_a \sum_b x(a, b) w(n - a, m - b)$$

```
[14]: X = training[0]
W = array([[1,1,1],[1,1,1], [1,1,1]])/9.

sfig = plt.figure()
plt.subplot(1,3,1)
plt.imshow(X, cmap='gray')
plt.title('x(n,m)')
plt.subplot(1,3,2)
plt.imshow(zeropad2(W, 5, 5, 5, 5), cmap='gray')
plt.title('w(n,m)')
plt.subplot(1,3,3)
plt.imshow(signal.convolve2d(X,W), cmap='gray')
plt.title('s(n,m)')
plt.close()
```

```
[15]: sfig
```

```
[15]:
```



9 Two Interpretations of Convolution

- Interpretation 1:
 - w is a filter on the frequency spectrum of signal x .
 - * Example filters: *low-pass, high-pass, band-pass, moving-average*
- Analysis is in the frequency domain:
 - **Time domain** $x(t) \iff$ **Frequency domain** $X(\omega)$
- Discrete-time Fourier transform (DTFT)
 - represent data as sum of complex exponentials basis functions with different frequencies.
 - * $e^{-i\omega t} = \cos(\omega t) - i \sin(\omega t)$
 - * Imaginary number: $i^2 = -1$
 - $X(\omega) = \sum_t x(t)e^{-i\omega t} \iff x(t) = \frac{1}{2\pi} \int X(\omega)e^{i\omega t} d\omega$
- **Key results:**
 - convolution in the time domain is equivalent to multiplication in the frequency domain:
 - * $s(t) = x(t) * w(t) \iff S(\omega) = X(\omega)W(\omega)$
 - we can design and analyze filters in the frequency domain, and then obtain their time-domain representation.

```
[16]: import numpy as np
from scipy.signal import butter, lfilter, freqz
import matplotlib.pyplot as plt
from scipy.fftpack import fft as fft

fs = 30.0          # sample rate, Hz

# Demonstrate the use of the filter.
# First make some data to be filtered.
T = 5.0           # seconds
n = int(T * fs)   # total number of samples
t = np.linspace(0, T, n, endpoint=False)
# "Noisy" data. We want to recover the 1.2 Hz signal from this.
```

```

data = np.sin(1.2*2*np.pi*t) + 1.5*np.cos(9*2*np.pi*t) + 0.5*np.sin(12.0*2*np.
    ↪pi*t)

tt = linspace(-10,10)
w = sin(3.*tt/2) / (tt/2);

y = convolve(data, w, mode='same')
ffig = plt.figure(figsize=(8,4))

plt.subplot(3, 2, 1)
plt.plot(range(0,n), data, 'b-', label='data')
plt.grid()
plt.title('original data  $x$')

plt.subplot(3, 2, 5)
plt.plot(range(0,n), y, 'g-', linewidth=2, label='filtered data')
plt.xlabel('time')
plt.title('filtered data  $s=x*w$')
plt.grid()

def calculateFFT(time,signal):
    signal = concatenate([signal, zeros((3*len(signal),))])
    N=len(signal)
    #df=1/((time[-1]-time[0]))
    #frequencies=[i*df for i in range(int(N/2.0))]
    frequencies = linspace(0,pi,int(N/2.))
    fftValues = [2.0/N*abs(i) for i in fft(signal,N)[0:int(N/2.0)] ]
    return frequencies,fftValues

plt.subplot(3, 2, 2)
originalfreqs,originalFFT=calculateFFT(t,data)
plt.plot(originalfreqs,originalFFT,"b",label="original")
plt.grid()
plt.title('original spectrum X')

plt.subplot(3, 2, 6)
filteredfreqs,filteredFFT=calculateFFT(t,y)
plt.plot(filteredfreqs,filteredFFT,"g",label="filtered")
plt.grid()
plt.title('filtered spectrum $S=X\cdot W$')
plt.xlabel('frequency')

```

```

# Plot the frequency response.
#w, h = freqz(b, a, worN=8000)
#plt.subplot(3, 2, 4)
##plt.plot(0.5*fs*w/np.pi, np.abs(h), 'b')
#plt.plot(cutoff, 0.5*np.sqrt(2), 'ko')
#plt.axvline(cutoff, color='k')
#plt.xlim(0, 0.5*fs)

plt.subplot(3,2,3)
plt.plot(tt,w, 'r')
plt.grid()
plt.title('filter $w$')
plt.subplot(3,2,4)
wfreqs,wFFT=calculateFFT(tt,w)
plt.plot(wfreqs,wFFT,"r",label="filtered")
plt.title("filter response $W$")
plt.grid()

#plt.subplots_adjust(hspace=0.45)
plt.tight_layout()
plt.close()

```

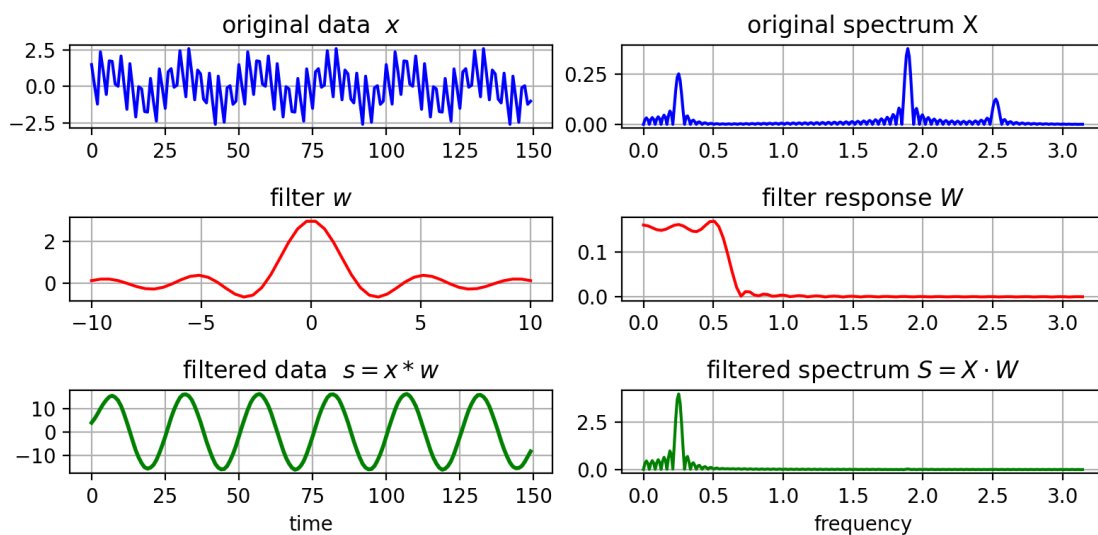
```

<>:56: SyntaxWarning: invalid escape sequence '\c'
<>:56: SyntaxWarning: invalid escape sequence '\c'
/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_74841/2482302383.py:5
6: SyntaxWarning: invalid escape sequence '\c'
    plt.title('filtered spectrum $S=X\cdot W$')

```

[17]: ffig

[17]:



- Relationships between time and frequency spectrum
 - short-duration (high-frequency) signal $\Leftrightarrow$ large frequency spectrum
 - long-duration (low-frequency) signal $\Leftrightarrow$ small frequency spectrum
- For filters...
 - short (small) filter only captures short-range correlations (high frequencies).
 - long (large) filter can capture long-range correlations (low frequencies).
- Interpretation 2:
 - $\tilde{w}$ is a pattern (template); try to find this pattern.
 - the maximum correlation occurs when pattern $\tilde{w}$ matches the local x .
 - * for a fixed energy $\|x\|^2 = 1$, $x = \frac{\tilde{w}}{\|\tilde{w}\|}$ has the maximum correlation with (response to) $\tilde{w}$.

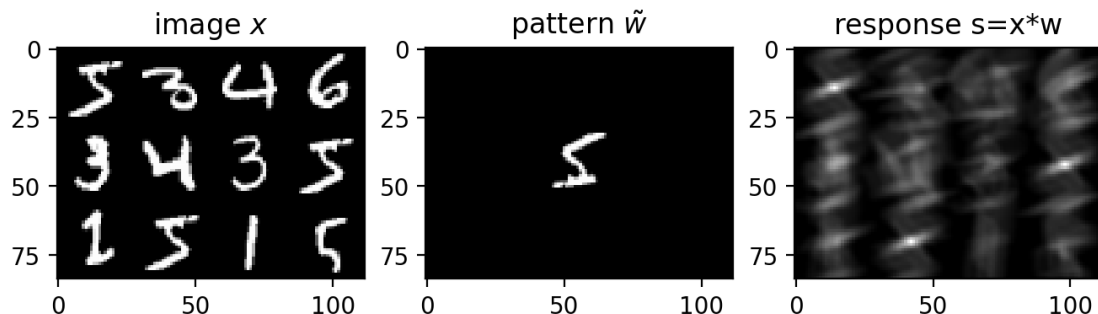
```
[18]: X = training[0].astype('float64')
W = flipplr(flipud(X.astype('float64'))))

XX = concatenate(
    [
        concatenate(training[[0,3,6,9]], axis=1),
        concatenate(training[[1,2,5,0]], axis=1),
        concatenate(training[[7,0,4,10]], axis=1)
    ],
    axis=0
).astype('float64')
```

```
[19]: pfig = plt.figure(figsize=(8,6))
plt.subplot(1,3,1)
plt.imshow(XX, cmap='gray')
plt.title('image $x$')
plt.subplot(1,3,2)
plt.imshow(zeropad2(W, 42, 42, 28, 28), cmap='gray')
plt.title('pattern $\tilde{w}$')
plt.subplot(1,3,3)
plt.imshow(signal.convolve2d(XX,W,mode='same'), cmap='gray')
plt.title('response $s=x*w$')
plt.close()
```

```
[20]: pfig
```

```
[20]:
```



10 Convolution as a layer

- output layer is the convolution of input with filter (kernel) $\mathbf{w}$.
 - $\mathbf{z} = \mathbf{x} * \mathbf{w}$.
- filter $\mathbf{w}$ acts locally on input $\mathbf{x}$.
 - $\mathbf{w}$ also called a **kernel**.
- Equivalent to a linear transformation (layer) where A has a particular form.
 - For example, if $\mathbf{w} = [w_1, w_2, w_3]$ and using “same” mode,

$$\mathbf{z} = \mathbf{x} * \mathbf{w}$$

$$= \mathbf{A}^T \mathbf{x} = \begin{bmatrix} w_2 & w_3 & 0 & 0 & 0 & 0 & \dots \\ w_1 & w_2 & w_3 & 0 & 0 & 0 & \dots \\ 0 & w_1 & w_2 & w_3 & 0 & 0 & \dots \\ 0 & 0 & w_1 & w_2 & w_3 & 0 & \dots \\ \dots & & & & & & \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_N \end{bmatrix}$$

– Note: $\mathbf{A}$ has size $|\mathbf{x}||\mathbf{z}|$, but only $|\mathbf{w}|$ parameters.

11 Translation equivariance

- Shifting x also shifts the response s .
- if $s = x * w$,
 - then $s(t - a) = x(t - a) * w(t)$
- We can find the pattern everywhere in x using the same filter.

```
[21]: X = training[0].astype('float64')
W = fliplr(flipud(X.astype('float64'))))

XX = concatenate(
    [
        concatenate(training[[7,7,7,7]], axis=1),
        concatenate(training[[7,0,7,7]], axis=1),
        concatenate(training[[7,7,7,7]], axis=1)
    ],
    axis=0
).astype('float64')

XX2 = concatenate(
    [
        concatenate(training[[7,7,7,7]], axis=1),
        concatenate(training[[7,7,0,7]], axis=1),
        concatenate(training[[7,7,7,7]], axis=1)
    ],
```

```

        axis=0
    ).astype('float64')

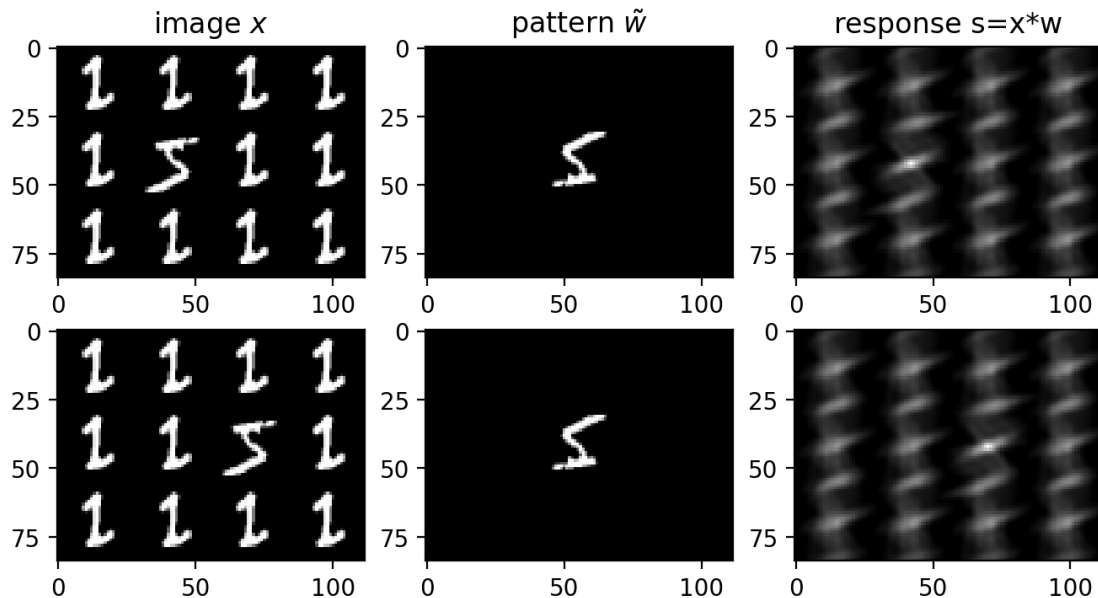
    cfig = plt.figure(figsize=(8,4))
    plt.subplot(2,3,1)
    plt.imshow(XX, cmap='gray')
    plt.title('image $x$')
    plt.subplot(2,3,2)
    plt.imshow(zeropad2(W, 42, 42, 28, 28), cmap='gray')
    plt.title('pattern $\tilde{w}$')
    plt.subplot(2,3,3)
    plt.imshow(signal.convolve2d(XX,W,mode='same'), cmap='gray')
    plt.title('response $s=x*w$')

    plt.subplot(2,3,4)
    plt.imshow(XX2, cmap='gray')
    plt.subplot(2,3,5)
    plt.imshow(zeropad2(W, 42, 42, 28, 28), cmap='gray')
    plt.subplot(2,3,6)
    plt.imshow(signal.convolve2d(XX2,W,mode='same'), cmap='gray')
    plt.close()

```

[22]: cfig

[22]:



12 Convolutional Neural Network (CNN)

- series of convolutional layers, sub-sampling layers, and MLP classifier.
 - convolutional and subsampling layers extract image features.
 - MLP uses extracted features for classification.

13 2D Convolution

- Use the spatial structure of the image
- 2D convolution filter
 - the weights $\mathbf{W}$ form a 2D filter template
 - filter response: $h = f(\sum_{x,y} W_{x,y} P_{x,y})$
 - * $\mathbf{P}$ is an image patch with the same size as $\mathbf{W}$.
- Convolution feature map
 - pass a sliding window over the image, and apply filter to get a *feature map*.
- Convolution modes
 - “**valid**” mode - only compute feature where convolution filter has valid image values.
 - * size of feature map is reduced.
- Convolution modes
 - “**same**” mode - zero-pad the border of the image
 - * feature map is the same size as the input image.
- Usually “same” is better since it looks at structures around border.
 - position information is implicitly encoded in the CNN features based on the zero-padding border.

14 2D Convolutional layer

- **Input:** HxW image with C channels
 - For example, in the first layer, C=3 for RGB channels.
 - defines a 3D volume: C x H x W (or H x W x C)
- **Features:** apply F convolution filters to get F feature maps.
 - Each feature map uses a 3D convolution filter (CxKxK) on the input
 - K is the spatial extent of the filter; total FCKK parameters
- **Activation:**
 - an activation function can be applied before output
- **Output:** a feature map with F channels
 - defines a 3D volume: F x H' x W'
 - H' and W' depend on various factors.

15 Combining Convolutional Layers

- Concatenate several convolutional layers.
 - From layer to layer
 - * spatial resolution decreases
 - * number of feature maps increases
 - Can extract high-level features in the final layers

- Feature map representation:

16 Spatial sub-sampling

- reduce the feature map size by subsampling feature maps between convolutional layers
 - *stride* for convolution filter - step size when moving the windows across the image.
- Spatial sub-sampling
 - *max-pooling layer* - use the maximum over a pooling window
 - * gathers features together, summarizes features in local region.
 - * introduces **translation invariance**
 - it doesn't matter where the maximal feature is located locally, it is passed to the next layer.
 - increases robustness to small changes in configuration of features

17 Receptive field size

- Stacking convolutional layers increases the effective size of the pattern filter
 - called **receptive field** - what pixels in the input affect a particular node.
 - larger receptive fields can see larger patterns.
 - Example: 2 convolutional layers
 - * $|\mathbf{w}_1| = 5, |\mathbf{w}_2| = 3$, receptive field size = $|\mathbf{w}_1| + |\mathbf{w}_2| - 1 = 7$

18 Advantages of Convolution Layers

- The convolutional filters extract the same features throughout the image.
 - Useful because the object can appear in different locations of the image (global translation equivariance).
- Pooling makes it robust to changes in feature configuration (local translation invariance).
- The number of parameters is small compared to Dense (Fully-connected) layer
 - Example: input is $C \times H \times W$, and output is $F \times H \times W$
 - * Number of MLP parameters: $(CHW+1) \times (FWH)$
 - * Number of CNN parameters: $F \times (CKK+1)$

19 Fully-connected layers (MLP)

- after several convolutional layers, input the feature map into an MLP to get the final classification.
- also called “fully-connected” (FC) layers.

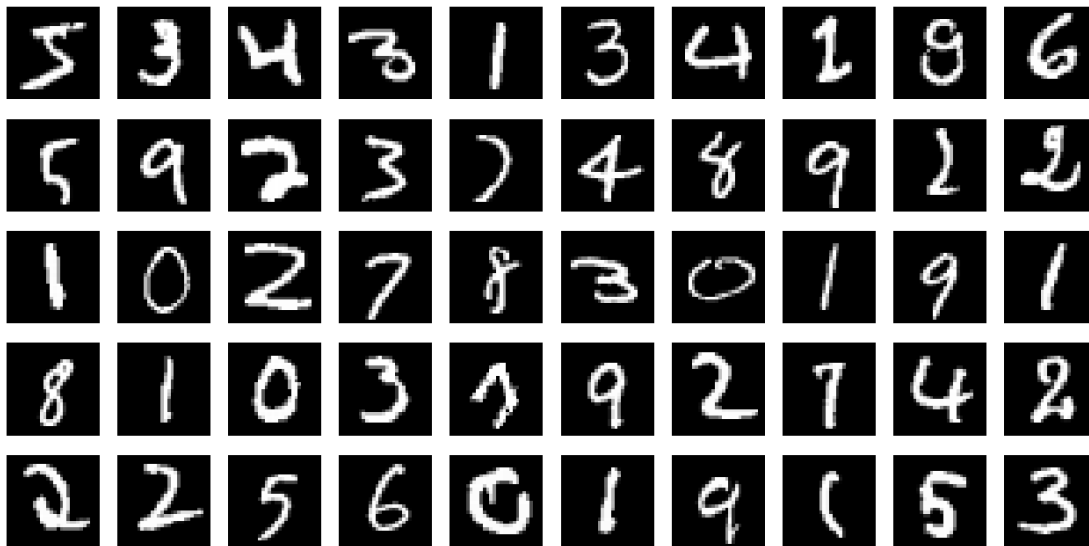
20 Example: Object classification CNN

- Each layer shows its feature maps for the example image.
 - early layers extract *low-level* (visual) features
 - * e.g., corners, edges
 - middle layers extract *mid-level* (part) features

- * e.g., object parts
- later layers extract *high-level* (semantic) features.
 - * e.g., object
- The number of feature channels increases with each layer
 - combining low-level parts together to get more higher-level parts
 - * e.g., {edges, corners} -> {wheel, door, window}
 - trading off spatial resolution for semantic specificity
 - * e.g., 512x512x3 RGB image -> 8x8x512 semantic features
- the spatial resolution decreases with each layer
 - increase the window size (receptive field) on the object
 - high-level semantic correspond to large regions.

21 CNN on MNIST

```
[23]: # Example images
plt.figure(figsize=(8,4))
show_imgs(training[0:50])
```



22 4D tensor format

- There are two common formats for the 4-D tensor:
 - “NCHW” - batch, channel, height, width - called 'channels_first'
 - “NHWC” - batch, height, width, channel - called 'channels_last'
- NCHW is required for PyTorch while NHWC is required for CPU version of Tensorflow 2.

23 Example on MNIST

- Pre-processing
 - scale to [0,1]
 - 4-D tensor: (sample, height, width, channel)
 - * channel = 1 (grayscale)
 - create training/validation sets

```
[24]: # Convert to PyTorch tensors and scale to 0-1
trainI = torch.from_numpy(training.reshape((6000, 1, 28, 28)) / 255.0).float()
testI = torch.from_numpy(testing.reshape((10000, 1, 28, 28)) / 255.0).float()

print(trainI.shape)
print(testI.shape)
```

```
torch.Size([6000, 1, 28, 28])
torch.Size([10000, 1, 28, 28])
```

- Generate fixed training and validation sets

```
[25]: # Generate fixed validation set
from sklearn.model_selection import train_test_split

vtrainI_np, validI_np, vtrainY_np, validY_np = train_test_split(
    trainI.numpy(), trainY,
    train_size=0.9, test_size=0.1, random_state=4487
)

vtrainI = torch.from_numpy(vtrainI_np).float()
validI = torch.from_numpy(validI_np).float()
vtrainY = torch.from_numpy(vtrainY_np).long()
validY = torch.from_numpy(validY_np).long()

print(f"Training set: {vtrainI.shape}, {vtrainY.shape}")
print(f"Validation set: {validI.shape}, {validY.shape}")

cid, cfreq = vtrainY.unique(return_counts=True)

print(f"Training set Distribution:", cfreq.numpy().tolist())
```

```
Training set: torch.Size([5400, 1, 28, 28]), torch.Size([5400])
Validation set: torch.Size([600, 1, 28, 28]), torch.Size([600])
Training set Distribution: [491, 599, 554, 556, 513, 518, 567, 584, 496, 522]
```

24 Shallow CNN Architecture

- 1 Convolution layer
 - 5x5x1 kernel, 10 features, *stride* = 2 (step-size between sliding windows)
 - No pooling here since the image input is small (28x28)

- Input: 28x28x1 (grayscale image) -> Output: 14x14x10
- 1 fully-connected layer (MLP), 50 nodes
 - Input: 14x14x10=1960 -> Output: 50
- Classification output node

```
[26]: # Build the network
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5, stride=2, padding=2) #
        ↪ input channels=1, output channels=10
        self.flatten = nn.Flatten()

        # Calculate the size after conv layers for the dense layers
        # Input: 28x28, after conv: (28/2)=14x14, flattened: 10*14*14 = 1960
        self.fc1 = nn.Linear(10 * 14 * 14, 50) # units=50
        self.fc2 = nn.Linear(50, 10) # units=10

        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        # Note: We'll use CrossEntropyLoss which includes softmax, so we don't
        ↪ apply it here
        return x
```

```
[27]: # Early stopping implementation
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.0001, verbose=1):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_accuracy = 0.0
        self.early_stop = False
        self.verbose = verbose

    def __call__(self, current_accuracy):
        if current_accuracy - self.best_accuracy > self.min_delta:
            self.best_accuracy = current_accuracy
            self.counter = 0
        else:
            self.counter += 1
            if self.verbose:
```

```

        print(f'EarlyStopping counter: {self.counter} out of {self.
↪patience}')
        if self.counter >= self.patience:
            self.early_stop = True
        return self.early_stop

```

```

[28]: # Model Parameter Summarization
def summarize_layer_parameters(model):
    """
    Summarizes the number of parameters in each layer of a PyTorch model.
    """
    trainable_table = PrettyTable(["Module Name", "Parameters"])
    untrainable_table = PrettyTable(["Module Name", "Parameters"])
    total_params = {'train':0, 'untrain':0}
    for name, parameter in model.named_parameters():
        if parameter.requires_grad: # Only count trainable parameters
            params = parameter.numel()
            trainable_table.add_row([name, params])
            total_params['train'] += params
        else: # Only count untrainable parameters
            params = parameter.numel()
            untrainable_table.add_row([name, params])
            total_params['untrain'] += params

    print(trainable_table)
    print(f"\nTotal Trainable Parameters: {total_params['train']}")
    if total_params['untrain'] > 0:
        print(untrainable_table)
    print(f"\nTotal Un-Trainable Parameters: {total_params['untrain']}")

```

```

[29]: # Model Evaluation
def evaluate_model(model, eval_loader, criterion=None):
    model.eval()
    eval_correct = 0
    eval_total = 0
    eval_loss = 0.0

    with torch.no_grad():
        for data, target in eval_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            _, predicted = torch.max(output.data, 1)
            eval_total += target.size(0)
            eval_correct += (predicted == target).sum().item()

        if criterion is not None:
            loss = criterion(output, target)

```

```

        eval_loss += loss.item()

    eval_accuracy = 100 * eval_correct / eval_total
    avg_eval_loss = eval_loss / len(eval_loader)

    return eval_accuracy if criterion is None else (eval_accuracy,
↪avg_eval_loss)

# Model Training
def train_model(model, train_loader, valid_loader, criterion, optimizer,
↪epochs=100, patience=5):
    history = {
        'train_loss': [], 'val_loss': [],
        'train_accuracy': [], 'val_accuracy': []
    }
    early_stopping = EarlyStopping()

    # best_val_loss = float('inf')
    # patience_counter = 0

    for epoch in range(epochs):
        # Training phase
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0

        for batch_idx, (data, target) in enumerate(train_loader):
            data, target = data.to(device), target.to(device)

            optimizer.zero_grad()
            output = model(data)

            # loss = criterion(output, torch.nn.functional.one_hot(target,
↪num_classes=10).float()) also works
            loss = criterion(output, target)

            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            _, predicted = torch.max(output.data, 1)
            train_total += target.size(0)
            train_correct += (predicted == target).sum().item()

        train_accuracy = 100 * train_correct / train_total

```

```

    avg_train_loss = train_loss / len(train_loader)

    # Validation phase
    val_accuracy, avg_val_loss = evaluate_model(model, valid_loader, criterion)

    # Store history
    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(avg_val_loss)
    history['train_accuracy'].append(train_accuracy / 100)
    history['val_accuracy'].append(val_accuracy / 100)

    train_msg = f' Train Loss: {avg_train_loss:.6f}, Train Acc: {train_accuracy:.2f}%;'
    valid_msg = f' Val Loss: {avg_val_loss:.6f}, Val Acc: {val_accuracy:.2f}%'

    print(f'Epoch {epoch+1}/{epochs}:', train_msg, valid_msg)

    # Check early stopping
    early_stopping(val_accuracy)
    if early_stopping.early_stop:
        print("Early stopping triggered")
        break
    else:
        torch.save(model.state_dict(), 'best_model.pth')

    # Load best model
    model.load_state_dict(torch.load('best_model.pth'))
    return history, model

```

```

[30]: # Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True

# Create model instance
model = SimpleCNN()

# Print model architecture
print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

# Prepare data - ensure correct shape for PyTorch (channel first)
# Assuming vtrainI, vtrainY, validI, validY are already defined as in previous
↳ cells

# Convert from (batch, height, width, channels) to (batch, channels, height,
↳ width)
if len(vtrainI.shape) == 4 and vtrainI.shape[-1] == 1: # If shape is (N, 28,
↳ 28, 1)
    vtrainI = vtrainI.transpose(0, 3, 1, 2)
    validI = validI.transpose(0, 3, 1, 2)

# Create DataLoaders
batch_size = 50
train_dataset = TensorDataset(vtrainI, vtrainY)
valid_dataset = TensorDataset(validI, validY)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Compile the network (equivalent in PyTorch)
criterion = nn.CrossEntropyLoss() # This includes softmax + cross entropy
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
↳ optimizer, epochs=100)

# Plot the training history
plot_history(history)

```

```

SimpleCNN(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=1960, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
)
+-----+-----+
| Module Name | Parameters |
+-----+-----+
| conv1.weight | 250 |

```


conv1.bias		10	
fc1.weight		98000	
fc1.bias		50	
fc2.weight		500	
fc2.bias		10	
+-----+			

Total Trainable Parameters: 98820

Total Un-Trainable Parameters: 0

Using device: cpu

Epoch 1/100: Train Loss: 1.000959, Train Acc: 68.33%; Val Loss: 0.427933, Val Acc: 86.50%

Epoch 2/100: Train Loss: 0.365901, Train Acc: 88.87%; Val Loss: 0.348270, Val Acc: 88.83%

Epoch 3/100: Train Loss: 0.279040, Train Acc: 91.91%; Val Loss: 0.242267, Val Acc: 92.67%

Epoch 4/100: Train Loss: 0.228215, Train Acc: 93.11%; Val Loss: 0.238575, Val Acc: 92.83%

Epoch 5/100: Train Loss: 0.183257, Train Acc: 94.59%; Val Loss: 0.213258, Val Acc: 93.67%

Epoch 6/100: Train Loss: 0.148498, Train Acc: 95.52%; Val Loss: 0.195522, Val Acc: 94.00%

Epoch 7/100: Train Loss: 0.119439, Train Acc: 96.28%; Val Loss: 0.180730, Val Acc: 95.00%

Epoch 8/100: Train Loss: 0.101738, Train Acc: 96.89%; Val Loss: 0.183262, Val Acc: 94.00%

EarlyStopping counter: 1 out of 5

Epoch 9/100: Train Loss: 0.082846, Train Acc: 97.48%; Val Loss: 0.214063, Val Acc: 94.00%

EarlyStopping counter: 2 out of 5

Epoch 10/100: Train Loss: 0.064539, Train Acc: 98.02%; Val Loss: 0.203230, Val Acc: 94.33%

EarlyStopping counter: 3 out of 5

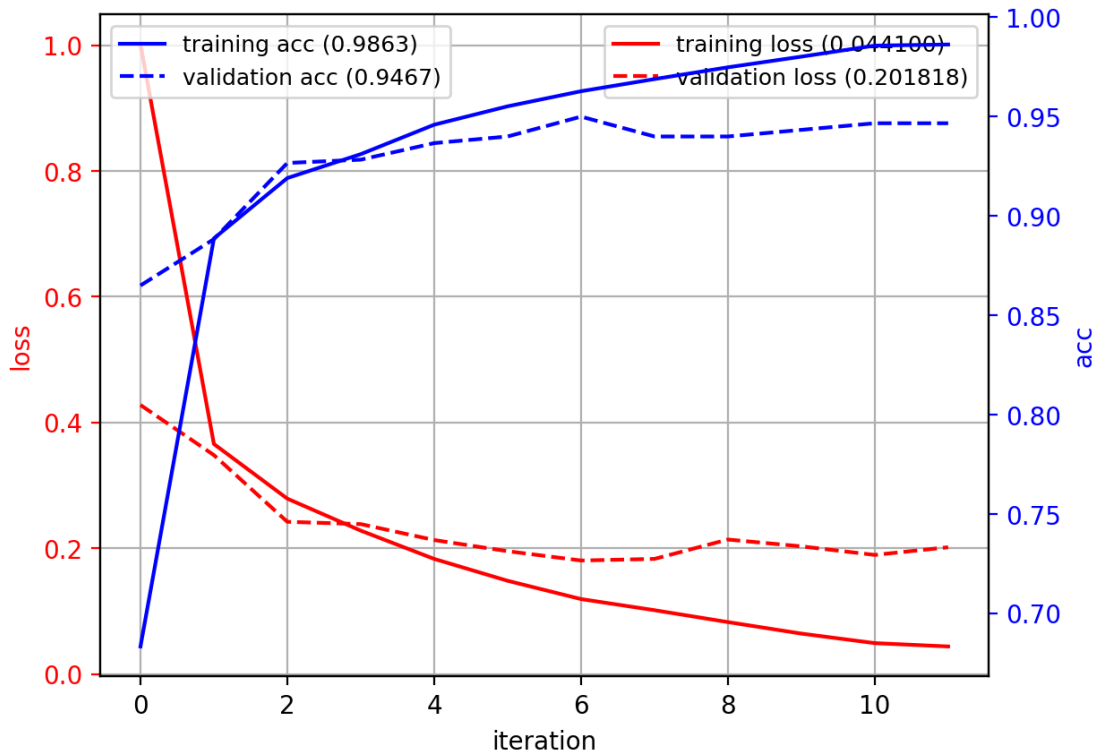
Epoch 11/100: Train Loss: 0.049383, Train Acc: 98.57%; Val Loss: 0.189592, Val Acc: 94.67%

EarlyStopping counter: 4 out of 5

Epoch 12/100: Train Loss: 0.044100, Train Acc: 98.63%; Val Loss: 0.201818, Val Acc: 94.67%

EarlyStopping counter: 5 out of 5

Early stopping triggered



```
[31]: from torch.utils.tensorboard import SummaryWriter
```

```
# Create a writer
```

```
writer = SummaryWriter()
```

```
# Add model graph to tensorboard
```

```
dummy_input = torch.randn(1, 1, 28, 28).to(device)
```

```
writer.add_graph(model, dummy_input)
```

```
writer.close()
```

```
print("Model graph saved to TensorBoard. Run: tensorboard --logdir=runs")
```

Model graph saved to TensorBoard. Run: tensorboard --logdir=runs

```
[32]: test_dataset = TensorDataset(testI, torch.from_numpy(testY).long())
```

```
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
acc = evaluate_model(model, test_loader)
```

```
print("test accuracy:", acc)
```

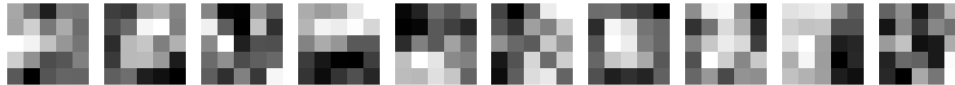
test accuracy: 94.22

- Visualize the convolutional filters

- filters are looking for local stroke features
 - * corners, edges, lines

```
[33]: W = model.conv1.weight.data.cpu()
print(W.shape)
flist = [squeeze(W[c]) for c in range(10)]
show_imgs(flist)
```

```
torch.Size([10, 1, 5, 5])
```



25 Deep CNN Architecture

- 3 Convolutional layers
 - 5x5x1 kernel, stride 2, 10 features (output 14x14x10)
 - 5x5x10 kernel, stride 2, 40 features (output 7x7x40)
 - 5x5x40 kernel, stride 1, 80 features (output 7x7x80)
 - set stride as 1 to avoid reducing the feature map too much)
- 1 fully-connected layer (7x7x80=3920 -> 50)
- Classification output node

```
[34]: class DeepCNN(nn.Module):
    def __init__(self, weight_decay=0.0001):
        super(DeepCNN, self).__init__()
        self.weight_decay = weight_decay

        self.conv1 = nn.Conv2d(1, 10, kernel_size=5, stride=2, padding=2)
        self.conv2 = nn.Conv2d(10, 40, kernel_size=5, stride=2, padding=2)
        self.conv3 = nn.Conv2d(40, 80, kernel_size=5, stride=1, padding=2)
        self.fc1 = nn.Linear(80 * 7 * 7, 50) # Adjusted based on input size,
        and strides
        self.fc2 = nn.Linear(50, 10)

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.relu(self.conv3(x))
        x = x.view(x.size(0), -1) # Flatten
```

```

        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = DeepCNN()

# Print model architecture
print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
                               optimizer, epochs=100)

# Plot the training history
plot_history(history)

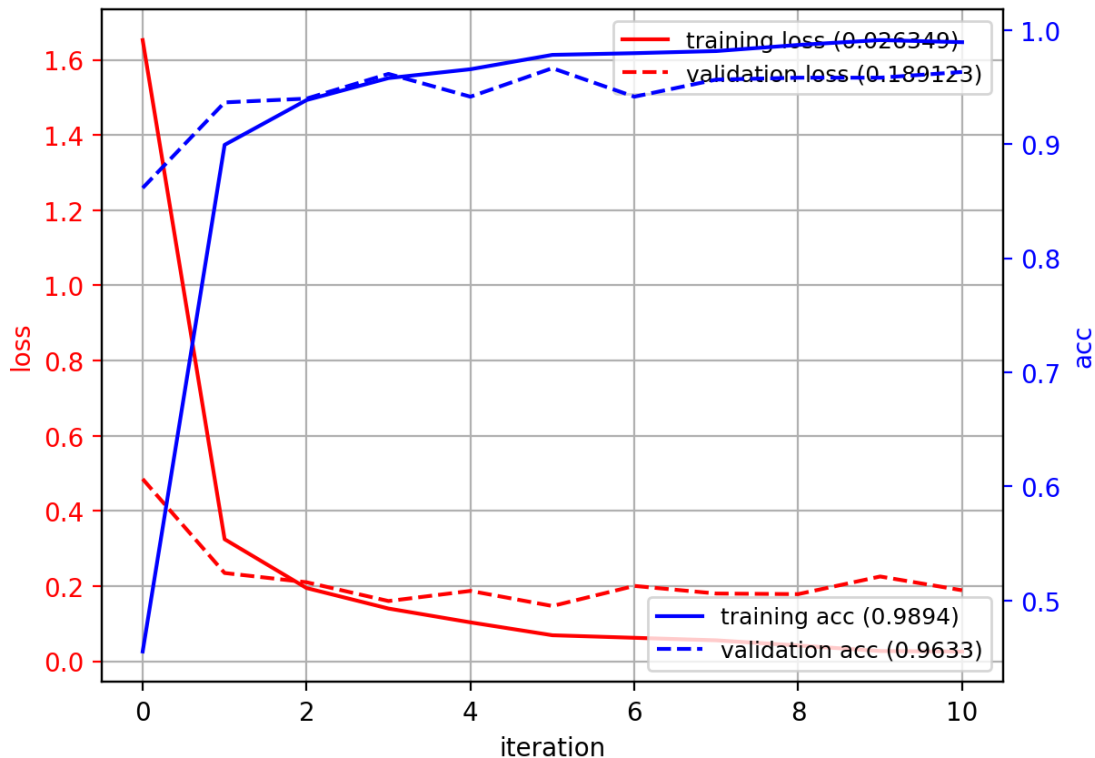
```

```

DeepCNN(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.5, inplace=False)
  (softmax): Softmax(dim=1)
)
Using device: cpu
Epoch 1/100:   Train Loss: 1.652544, Train Acc: 45.54%;   Val Loss: 0.485118,
Val Acc: 86.17%
Epoch 2/100:   Train Loss: 0.324927, Train Acc: 89.94%;   Val Loss: 0.235094,
Val Acc: 93.67%
Epoch 3/100:   Train Loss: 0.195066, Train Acc: 93.87%;   Val Loss: 0.210481,
Val Acc: 94.00%
Epoch 4/100:   Train Loss: 0.140573, Train Acc: 95.80%;   Val Loss: 0.160527,
Val Acc: 96.17%
Epoch 5/100:   Train Loss: 0.103770, Train Acc: 96.57%;   Val Loss: 0.187480,
Val Acc: 94.17%
EarlyStopping counter: 1 out of 5
Epoch 6/100:   Train Loss: 0.069524, Train Acc: 97.83%;   Val Loss: 0.147277,
Val Acc: 96.67%

```

Epoch 7/100: Train Loss: 0.062744, Train Acc: 97.98%; Val Loss: 0.200751, Val Acc: 94.17%
 EarlyStopping counter: 1 out of 5
 Epoch 8/100: Train Loss: 0.056061, Train Acc: 98.17%; Val Loss: 0.180467, Val Acc: 95.67%
 EarlyStopping counter: 2 out of 5
 Epoch 9/100: Train Loss: 0.042313, Train Acc: 98.70%; Val Loss: 0.178956, Val Acc: 95.83%
 EarlyStopping counter: 3 out of 5
 Epoch 10/100: Train Loss: 0.027585, Train Acc: 99.13%; Val Loss: 0.225759, Val Acc: 95.83%
 EarlyStopping counter: 4 out of 5
 Epoch 11/100: Train Loss: 0.026349, Train Acc: 98.94%; Val Loss: 0.189123, Val Acc: 96.33%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered



```
[35]: acc = evaluate_model(model, test_loader)
      print("test accuracy: ", acc)
```

test accuracy: 95.97

26 Summary

- Convolution operation
 - looks at the local structure of the signal (1D, 2D, 3D, etc).
 - two interpretations: filtering, pattern matching
- Convolutional neural network (CNN)
 - convolutional layer - use convolution instead of dense connections
 - learns to extract image features, and learns classifier.

Lecture6b

October 5, 2025

1 CS5489 - Machine Learning

2 Lecture 6b - Regularization

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

- Convolutional neural network (CNN)
- Regularization

```
[1]: # setup
import os
%matplotlib inline
# matplotlib.use('Agg')
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
import numpy as np
from numpy import *
from sklearn import *
from scipy import stats
from scipy import signal
from prettytable import PrettyTable

rbow = plt.get_cmap('rainbow')
```

```
[2]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from torch.utils.data import DataLoader, TensorDataset, Dataset
from torch.utils.tensorboard import SummaryWriter
import struct
import sys
print(f"Python: {sys.version} PyTorch: {torch.__version__}")
```

```
# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Python: 3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 16:09:02) [GCC 11.2.0] PyTorch: 2.8.0+cu129
Using device: cuda

```
[3]: def plot_history(history):
    fig, ax1 = plt.subplots()

    ax1.plot(history['train_loss'], 'r', label="training loss ({:.6f})".
    ↪format(history['train_loss'][-1]))
    ax1.plot(history['val_loss'], 'r--', label="validation loss ({:.6f})".
    ↪format(history['val_loss'][-1]))
    ax1.grid(True)
    ax1.set_xlabel('iteration')
    ax1.legend(loc="best", fontsize=9)
    ax1.set_ylabel('loss', color='r')
    ax1.tick_params('y', colors='r')

    if 'train_accuracy' in history:
        ax2 = ax1.twinx()

        ax2.plot(history['train_accuracy'], 'b', label="training acc ({:.4f})".
        ↪format(history['train_accuracy'][-1]))
        ax2.plot(history['val_accuracy'], 'b--', label="validation acc ({:.
        ↪4f})".format(history['val_accuracy'][-1]))

        ax2.legend(loc="best", fontsize=9)
        ax2.set_ylabel('acc', color='b')
        ax2.tick_params('y', colors='b')
    plt.show()
```

```
[4]: def show_imgs(W_list, nc=10, highlight_green=None, highlight_red=None,
    ↪titles=None):
    nfilter = len(W_list)
    nr = (nfilter - 1) // nc + 1
    for i in range(nr):
        for j in range(nc):
            idx = i * nc + j
            if idx == nfilter:
                break
            plt.subplot(nr, nc, idx + 1)
            cur_W = W_list[idx]
            plt.imshow(cur_W, cmap='gray', interpolation='nearest')
```



```

    if titles is not None:
        plt.title(titles % idx)

    if ((highlight_green is not None) and highlight_green[idx]) or \
        ((highlight_red is not None) and highlight_red[idx]):
        ax = plt.gca()
        if highlight_green[idx]:
            mycol = '#00FF00'
        else:
            mycol = 'r'
        for S in ['bottom', 'top', 'right', 'left']:
            ax.spines[S].set_color(mycol)
            ax.spines[S].set_lw(2.0)
            ax.xaxis.set_ticks_position('none')
            ax.yaxis.set_ticks_position('none')
            ax.set_xticks([])
            ax.set_yticks([])
        else:
            plt.gca().set_axis_off()

```

```

[5]: def read_32int(f):
    return struct.unpack('>I', f.read(4))[0]
def read_img(img_path):
    with open(img_path, 'rb') as f:
        magic_num = read_32int(f)
        num_image = read_32int(f)
        n_row = read_32int(f)
        n_col = read_32int(f)
        #print 'num_image = {}; n_row = {}; n_col = {}'.format(num_image,
↪n_row, n_col)
        res = []
        npixel = n_row * n_col
        res_arr = fromfile(f, dtype='B')
        res_arr = res_arr.reshape((num_image, n_row, n_col), order='C')
        #print 'image data shape = {}'.format(res_arr.shape)
        return num_image, n_row, n_col, res_arr
def read_label(label_path):
    with open(label_path, 'rb') as f:
        magic_num = read_32int(f)
        num_label = read_32int(f)
        #print 'num_label = {}'.format(num_label)
        res_arr = fromfile(f, dtype='B')
        #print res_arr.shape
        #res_arr = res_arr.reshape((num_label, 1))
        res_arr = res_arr.ravel()
        #print 'label data shape = {}'.format(res_arr.shape)
        return num_label, res_arr

```

```
[6]: n_train, nrow, ncol, training = read_img('data/train-images.idx3-ubyte')
_, trainY = read_label('data/train-labels.idx1-ubyte')
n_test, _, _, testing = read_img('data/t10k-images.idx3-ubyte')
_, testY = read_label('data/t10k-labels.idx1-ubyte')

# for demonstration we only use 10% of the training data
sample_index = range(0, training.shape[0], 10)
training = training[sample_index]
trainY = trainY[sample_index]
print(training.shape)
print(trainY.shape)
print(testing.shape)
print(testY.shape)

(6000, 28, 28)
(6000,)
(10000, 28, 28)
(10000,)

[7]: # Convert to PyTorch tensors and scale to 0-1
trainI = torch.from_numpy(training.reshape((6000, 1, 28, 28)) / 255.0).float()
testI = torch.from_numpy(testing.reshape((10000, 1, 28, 28)) / 255.0).float()

print(trainI.shape)
print(testI.shape)

torch.Size([6000, 1, 28, 28])
torch.Size([10000, 1, 28, 28])

[8]: # Generate fixed validation set
from sklearn.model_selection import train_test_split

vtrainI_np, validI_np, vtrainY_np, validY_np = train_test_split(
    trainI.numpy(), trainY,
    train_size=0.9, test_size=0.1, random_state=4487
)

vtrainI = torch.from_numpy(vtrainI_np).float()
validI = torch.from_numpy(validI_np).float()
vtrainY = torch.from_numpy(vtrainY_np).long()
validY = torch.from_numpy(validY_np).long()

print(f"Training set: {vtrainI.shape}, {vtrainY.shape}")
print(f"Validation set: {validI.shape}, {validY.shape}")

cid, cfreq = vtrainY.unique(return_counts=True)
```

```
print(f"Training set Distribution:", cfreq.numpy().tolist())
```

Training set: torch.Size([5400, 1, 28, 28]), torch.Size([5400])

Validation set: torch.Size([600, 1, 28, 28]), torch.Size([600])

Training set Distribution: [491, 599, 554, 556, 513, 518, 567, 584, 496, 522]

4 Regularization with L2-norm

- One way to regularize the network is to add an L2-norm penalty on the weights
 - Add a penalty term to the loss function
 - * larger weights have higher penalty
 - * $\hat{L} = L + \lambda \frac{1}{2} \|\mathbf{w}\|^2$
- Taking the gradient of $\hat{L}$,
 - $\frac{d\hat{L}}{d\mathbf{w}} = \frac{dL}{d\mathbf{w}} + \lambda \mathbf{w}$
- Applying gradient descent
 - $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{d\hat{L}}{d\mathbf{w}}$
 - $\mathbf{w} \leftarrow \mathbf{w} - \eta (\frac{dL}{d\mathbf{w}} + \lambda \mathbf{w})$
 - $\mathbf{w} \leftarrow (1 - \eta\lambda) \mathbf{w} - \eta \frac{dL}{d\mathbf{w}}$
- the weights shrink by $(1 - \eta\lambda)$ in each iteration of gradient descent.
 - prevents the $\mathbf{w}$ from getting too large.
 - Sometimes this is called **weight-decay** regularization.
- The optimal solution is an equilibrium point between L2-norm (dashed-line) and the loss (solid-line).
 - $\lambda \mathbf{w} = -\frac{dL}{d\mathbf{w}}$.

5 Example: L2 regularization

- Apply L2 regularization on all layers.
 - `kernel_regularizer` keyword

```
[9]: class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.conv1 = nn.Conv2d(1, 10, kernel_size=5, stride=2, padding=2)
        self.conv2 = nn.Conv2d(10, 40, kernel_size=5, stride=2, padding=2)
        self.conv3 = nn.Conv2d(40, 80, kernel_size=5, stride=1, padding=2)
        self.fc1 = nn.Linear(80 * 7 * 7, 50) # Adjusted based on input size
        and strides
        self.fc2 = nn.Linear(50, 10)

        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
```

```

        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.relu(self.conv3(x))
        x = x.view(x.size(0), -1)  # Flatten
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

    def l2_regularization(self):
        return 0.0

class CNNWithL2(CNN):
    def __init__(self, weight_decay=0.0001):
        super(CNNWithL2, self).__init__()
        self.weight_decay = weight_decay

    def l2_regularization(self):
        l2_reg = torch.tensor(0.).to(device)
        for param in self.parameters():
            l2_reg += torch.norm(param, p=2)
        return self.weight_decay * 0.5 * l2_reg

```

```

[10]: # Early stopping implementation
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.0001, verbose=1):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_accuracy = 0.0
        self.early_stop = False
        self.verbose = verbose

    def __call__(self, current_accuracy):
        if current_accuracy - self.best_accuracy > self.min_delta:
            self.best_accuracy = current_accuracy
            self.counter = 0
        else:
            self.counter += 1
            if self.verbose:
                print(f'EarlyStopping counter: {self.counter} out of {self.
↪patience}')
            if self.counter >= self.patience:
                self.early_stop = True
        return self.early_stop

```

```

[11]: # Model Parameter Summarization
def summarize_layer_parameters(model):

```

```

"""
Summarizes the number of parameters in each layer of a PyTorch model.
"""

trainable_table = PrettyTable(["Module Name", "Parameters"])
untrainable_table = PrettyTable(["Module Name", "Parameters"])
total_params = {'train':0, 'untrain':0}
for name, parameter in model.named_parameters():
    if parameter.requires_grad: # Only count trainable parameters
        params = parameter.numel()
        trainable_table.add_row([name, params])
        total_params['train'] += params
    else: # Only count untrainable parameters
        params = parameter.numel()
        untrainable_table.add_row([name, params])
        total_params['untrain'] += params

print(trainable_table)
print(f"\nTotal Trainable Parameters: {total_params['train']}")
if total_params['untrain'] > 0:
    print(untrainable_table)
print(f"\nTotal Un-Trainable Parameters: {total_params['untrain']}")

```

```

[12]: # Model Evaluation
def evaluate_model(model, eval_loader, criterion=None):
    model.eval()
    eval_correct = 0
    eval_total = 0
    eval_loss = 0.0

    with torch.no_grad():
        for data, target in eval_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            _, predicted = torch.max(output.data, 1)
            eval_total += target.size(0)
            eval_correct += (predicted == target).sum().item()

            if criterion is not None:
                loss = criterion(output, target)
                eval_loss += loss.item()

    eval_accuracy = 100 * eval_correct / eval_total
    avg_eval_loss = eval_loss / len(eval_loader)

    return eval_accuracy if criterion is None else (eval_accuracy,
↪avg_eval_loss)

```

```

# Model Training
def train_model(model, train_loader, valid_loader, criterion, optimizer,
    epochs=100, patience=5, writer='runs/cnn'):
    history = {
        'train_loss': [], 'val_loss': [],
        'train_accuracy': [], 'val_accuracy': []
    }
    early_stopping = EarlyStopping(patience=patience)
    ckpt_path = os.path.join(os.path.abspath('.'), writer, 'best_model.pth')
    writer = SummaryWriter(writer)

    # best_val_loss = float('inf')
    # patience_counter = 0

    for epoch in range(epochs):
        # Training phase
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0

        for batch_idx, (data, target) in enumerate(train_loader):
            data, target = data.to(device), target.to(device)

            optimizer.zero_grad()
            output = model(data)

            # loss = criterion(output, torch.nn.functional.one_hot(target,
    num_classes=10).float()) also works
            loss = criterion(output, target) + model.l2_regularization()

            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            _, predicted = torch.max(output.data, 1)
            train_total += target.size(0)
            train_correct += (predicted == target).sum().item()

        train_accuracy = 100 * train_correct / train_total
        avg_train_loss = train_loss / len(train_loader)

        # Validation phase
        val_accuracy, avg_val_loss =
    evaluate_model(model, valid_loader, criterion)

```

```

    # Store history
    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(avg_val_loss)
    history['train_accuracy'].append(train_accuracy / 100)
    history['val_accuracy'].append(val_accuracy / 100)

    writer.add_scalar("loss/train", avg_train_loss, epoch)
    writer.add_scalar("loss/valid", avg_val_loss, epoch)
    writer.add_scalar("acc/train", train_accuracy / 100.0, epoch)
    writer.add_scalar("acc/valid", val_accuracy / 100.0, epoch)

    train_msg = f' Train Loss: {avg_train_loss:.6f}, Train Acc: {train_accuracy:.2f}%;'
    valid_msg = f' Val Loss: {avg_val_loss:.6f}, Val Acc: {val_accuracy:.2f}%;'

    print(f'Epoch {epoch+1}/{epochs}:', train_msg, valid_msg)

    # Check early stopping
    if patience > 0:
        early_stopping(val_accuracy)
        if early_stopping.early_stop:
            print("Early stopping triggered")
            break
        else:
            torch.save(model.state_dict(), ckpt_path)
    else:
        torch.save(model.state_dict(), ckpt_path)

    # Load best model
    model.load_state_dict(torch.load(ckpt_path))
    writer.flush()
    writer.close()
    return history, model

```

```

[13]: # Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True

# Create model instance
model = CNNWithL2()

# Print model architecture

```

```

print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

# Prepare data - ensure correct shape for PyTorch (channel first)
# Assuming vtrainI, vtrainY, validI, validY are already defined as in previous
# cells

# Convert from (batch, height, width, channels) to (batch, channels, height,
# width)
if len(vtrainI.shape) == 4 and vtrainI.shape[-1] == 1: # If shape is (N, 28,
# 28, 1)
    vtrainI = vtrainI.transpose(0, 3, 1, 2)
    validI = validI.transpose(0, 3, 1, 2)

# Create DataLoaders
batch_size = 50
train_dataset = TensorDataset(vtrainI, vtrainY)
valid_dataset = TensorDataset(validI, validY)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Compile the network (equivalent in PyTorch)
criterion = nn.CrossEntropyLoss() # This includes softmax + cross entropy
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
# optimizer, epochs=100, writer='runs/l2reg')

# Plot the training history
plot_history(history)

```

```

CNNWithL2(
    (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
    (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
    (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (fc1): Linear(in_features=3920, out_features=50, bias=True)
    (fc2): Linear(in_features=50, out_features=10, bias=True)
    (relu): ReLU()
    (softmax): Softmax(dim=1)

```


)

+-----+-----+		
Module Name	Parameters	
+-----+-----+		
conv1.weight	250	
conv1.bias	10	
conv2.weight	10000	
conv2.bias	40	
conv3.weight	80000	
conv3.bias	80	
fc1.weight	196000	
fc1.bias	50	
fc2.weight	500	
fc2.bias	10	
+-----+-----+		

Total Trainable Parameters: 286940

Total Un-Trainable Parameters: 0

Using device: cuda

Epoch 1/100: Train Loss: 1.620371, Train Acc: 44.61%; Val Loss: 0.468100,
Val Acc: 84.33%

Epoch 2/100: Train Loss: 0.329343, Train Acc: 90.11%; Val Loss: 0.245200,
Val Acc: 93.17%

Epoch 3/100: Train Loss: 0.203770, Train Acc: 93.59%; Val Loss: 0.214860,
Val Acc: 93.00%

EarlyStopping counter: 1 out of 5

Epoch 4/100: Train Loss: 0.133710, Train Acc: 95.72%; Val Loss: 0.158682,
Val Acc: 95.17%

Epoch 5/100: Train Loss: 0.095974, Train Acc: 96.89%; Val Loss: 0.199730,
Val Acc: 94.50%

EarlyStopping counter: 1 out of 5

Epoch 6/100: Train Loss: 0.082481, Train Acc: 97.37%; Val Loss: 0.145677,
Val Acc: 95.83%

Epoch 7/100: Train Loss: 0.060736, Train Acc: 98.00%; Val Loss: 0.149765,
Val Acc: 95.83%

EarlyStopping counter: 1 out of 5

Epoch 8/100: Train Loss: 0.040754, Train Acc: 98.78%; Val Loss: 0.132005,
Val Acc: 96.17%

Epoch 9/100: Train Loss: 0.032565, Train Acc: 98.91%; Val Loss: 0.176460,
Val Acc: 95.50%

EarlyStopping counter: 1 out of 5

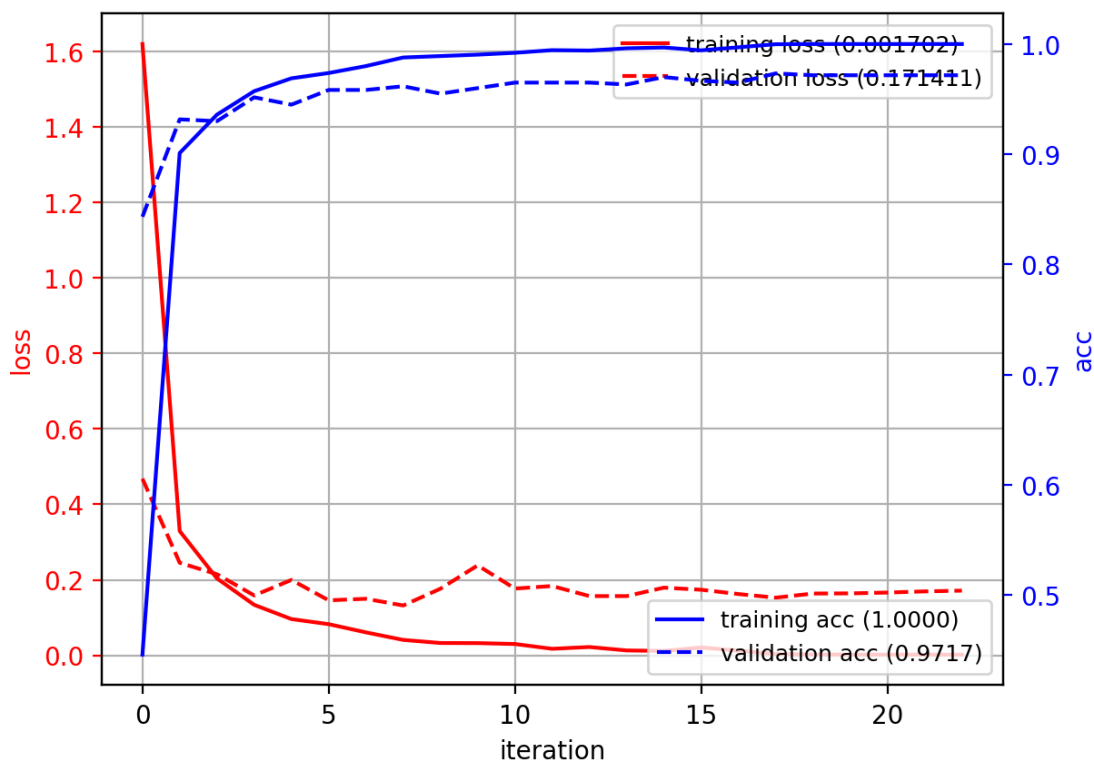
Epoch 10/100: Train Loss: 0.032279, Train Acc: 99.04%; Val Loss: 0.238308,
Val Acc: 96.00%

EarlyStopping counter: 2 out of 5

Epoch 11/100: Train Loss: 0.029881, Train Acc: 99.20%; Val Loss: 0.176865,
Val Acc: 96.50%

Epoch 12/100: Train Loss: 0.017214, Train Acc: 99.44%; Val Loss: 0.183611,

Val Acc: 96.50%
EarlyStopping counter: 1 out of 5
Epoch 13/100: Train Loss: 0.021913, Train Acc: 99.41%; Val Loss: 0.156942,
Val Acc: 96.50%
EarlyStopping counter: 2 out of 5
Epoch 14/100: Train Loss: 0.012814, Train Acc: 99.61%; Val Loss: 0.156906,
Val Acc: 96.33%
EarlyStopping counter: 3 out of 5
Epoch 15/100: Train Loss: 0.011452, Train Acc: 99.69%; Val Loss: 0.179287,
Val Acc: 97.00%
Epoch 16/100: Train Loss: 0.020812, Train Acc: 99.43%; Val Loss: 0.173752,
Val Acc: 96.67%
EarlyStopping counter: 1 out of 5
Epoch 17/100: Train Loss: 0.012141, Train Acc: 99.70%; Val Loss: 0.162341,
Val Acc: 96.50%
EarlyStopping counter: 2 out of 5
Epoch 18/100: Train Loss: 0.003358, Train Acc: 99.98%; Val Loss: 0.152713,
Val Acc: 97.33%
Epoch 19/100: Train Loss: 0.001990, Train Acc: 100.00%; Val Loss: 0.163681,
Val Acc: 97.17%
EarlyStopping counter: 1 out of 5
Epoch 20/100: Train Loss: 0.001826, Train Acc: 100.00%; Val Loss: 0.164104,
Val Acc: 97.17%
EarlyStopping counter: 2 out of 5
Epoch 21/100: Train Loss: 0.001752, Train Acc: 100.00%; Val Loss: 0.166261,
Val Acc: 97.17%
EarlyStopping counter: 3 out of 5
Epoch 22/100: Train Loss: 0.001722, Train Acc: 100.00%; Val Loss: 0.169351,
Val Acc: 97.17%
EarlyStopping counter: 4 out of 5
Epoch 23/100: Train Loss: 0.001702, Train Acc: 100.00%; Val Loss: 0.171411,
Val Acc: 97.17%
EarlyStopping counter: 5 out of 5
Early stopping triggered



```
[14]: test_dataset = TensorDataset(testI, torch.from_numpy(testY).long())
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
acc = evaluate_model(model, test_loader)

print("test accuracy:", acc)
```

test accuracy: 96.58

6 Which layers should we apply L2 regularization?

- Consider the following network: $f(\mathbf{x}) = \mathbf{A}^T r(\mathbf{B}^T \mathbf{x})$
 - $r(\cdot)$ is the ReLU activation.
- Consider the scaling value $\epsilon > 0$:

$$f(\mathbf{x}) = \mathbf{A}^T r(\mathbf{B}^T \mathbf{x}) = \mathbf{A}^T r\left(\frac{\epsilon}{\epsilon} \mathbf{B}^T \mathbf{x}\right) = \frac{1}{\epsilon} \mathbf{A}^T r(\epsilon \mathbf{B}^T \mathbf{x})$$

– the last line is because for $b > 0$, $r(ba) = \max(0, ba) = b \max(0, a)$

- If we decrease $\mathbf{B}$ by $\epsilon < 1$, we can increase $\mathbf{A}$ correspondingly to create the same network (with the same loss L).
 - $\mathbf{B} \rightarrow \epsilon \mathbf{B}$
 - $\mathbf{A} \rightarrow \frac{1}{\epsilon} \mathbf{A}$

- Thus, when applying L2 regularization on only one layer (e.g., **B**), the other layer (e.g., **A**) can be adjusted to get the same network.
 - The L2-norm is artificially reduced without changing the network function.
 - i.e., it has no actual effect.
- **Answer:** apply L2 regularization to *all* layers!

7 Overfitting

- occurs when validation curve starts to increase
 - network is specializing only on the training data, losing generalization ability.

```
[15]: model = CNN()

print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
                               optimizer, epochs=100, patience=-1, writer='runs/overfit')
```

```
CNN(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
)
Using device: cuda
Epoch 1/100:   Train Loss: 1.681100, Train Acc: 45.15%;   Val Loss: 0.422775,
Val Acc: 86.50%
Epoch 2/100:   Train Loss: 0.326791, Train Acc: 90.52%;   Val Loss: 0.228414,
Val Acc: 92.83%
Epoch 3/100:   Train Loss: 0.204111, Train Acc: 93.91%;   Val Loss: 0.165633,
Val Acc: 95.33%
Epoch 4/100:   Train Loss: 0.131937, Train Acc: 95.72%;   Val Loss: 0.189064,
Val Acc: 94.50%
Epoch 5/100:   Train Loss: 0.099432, Train Acc: 96.96%;   Val Loss: 0.140649,
Val Acc: 95.67%
Epoch 6/100:   Train Loss: 0.072953, Train Acc: 97.65%;   Val Loss: 0.131709,
```

Val Acc: 97.00%
 Epoch 7/100: Train Loss: 0.051284, Train Acc: 98.43%; Val Loss: 0.172873,
 Val Acc: 96.17%
 Epoch 8/100: Train Loss: 0.043922, Train Acc: 98.67%; Val Loss: 0.145643,
 Val Acc: 96.00%
 Epoch 9/100: Train Loss: 0.031168, Train Acc: 98.93%; Val Loss: 0.160849,
 Val Acc: 96.17%
 Epoch 10/100: Train Loss: 0.024159, Train Acc: 99.22%; Val Loss: 0.184941,
 Val Acc: 96.17%
 Epoch 11/100: Train Loss: 0.030437, Train Acc: 98.94%; Val Loss: 0.180374,
 Val Acc: 97.00%
 Epoch 12/100: Train Loss: 0.021612, Train Acc: 99.24%; Val Loss: 0.219290,
 Val Acc: 96.00%
 Epoch 13/100: Train Loss: 0.014712, Train Acc: 99.59%; Val Loss: 0.176683,
 Val Acc: 96.67%
 Epoch 14/100: Train Loss: 0.006732, Train Acc: 99.87%; Val Loss: 0.198587,
 Val Acc: 96.67%
 Epoch 15/100: Train Loss: 0.003014, Train Acc: 99.94%; Val Loss: 0.229721,
 Val Acc: 96.67%
 Epoch 16/100: Train Loss: 0.001291, Train Acc: 99.96%; Val Loss: 0.216829,
 Val Acc: 97.00%
 Epoch 17/100: Train Loss: 0.001173, Train Acc: 99.94%; Val Loss: 0.217470,
 Val Acc: 97.00%
 Epoch 18/100: Train Loss: 0.000271, Train Acc: 100.00%; Val Loss: 0.224117,
 Val Acc: 97.33%
 Epoch 19/100: Train Loss: 0.000186, Train Acc: 100.00%; Val Loss: 0.227993,
 Val Acc: 97.50%
 Epoch 20/100: Train Loss: 0.000153, Train Acc: 100.00%; Val Loss: 0.231361,
 Val Acc: 97.50%
 Epoch 21/100: Train Loss: 0.000131, Train Acc: 100.00%; Val Loss: 0.235502,
 Val Acc: 97.50%
 Epoch 22/100: Train Loss: 0.000116, Train Acc: 100.00%; Val Loss: 0.237838,
 Val Acc: 97.50%
 Epoch 23/100: Train Loss: 0.000104, Train Acc: 100.00%; Val Loss: 0.240231,
 Val Acc: 97.50%
 Epoch 24/100: Train Loss: 0.000095, Train Acc: 100.00%; Val Loss: 0.242329,
 Val Acc: 97.50%
 Epoch 25/100: Train Loss: 0.000087, Train Acc: 100.00%; Val Loss: 0.244578,
 Val Acc: 97.50%
 Epoch 26/100: Train Loss: 0.000080, Train Acc: 100.00%; Val Loss: 0.246407,
 Val Acc: 97.50%
 Epoch 27/100: Train Loss: 0.000075, Train Acc: 100.00%; Val Loss: 0.248349,
 Val Acc: 97.50%
 Epoch 28/100: Train Loss: 0.000071, Train Acc: 100.00%; Val Loss: 0.249632,
 Val Acc: 97.50%
 Epoch 29/100: Train Loss: 0.000066, Train Acc: 100.00%; Val Loss: 0.251564,
 Val Acc: 97.50%
 Epoch 30/100: Train Loss: 0.000062, Train Acc: 100.00%; Val Loss: 0.252831,

Val Acc: 97.50%
 Epoch 31/100: Train Loss: 0.000059, Train Acc: 100.00%; Val Loss: 0.253965,
 Val Acc: 97.50%
 Epoch 32/100: Train Loss: 0.000056, Train Acc: 100.00%; Val Loss: 0.255242,
 Val Acc: 97.50%
 Epoch 33/100: Train Loss: 0.000053, Train Acc: 100.00%; Val Loss: 0.256716,
 Val Acc: 97.50%
 Epoch 34/100: Train Loss: 0.000051, Train Acc: 100.00%; Val Loss: 0.257665,
 Val Acc: 97.50%
 Epoch 35/100: Train Loss: 0.000048, Train Acc: 100.00%; Val Loss: 0.259044,
 Val Acc: 97.50%
 Epoch 36/100: Train Loss: 0.000046, Train Acc: 100.00%; Val Loss: 0.259916,
 Val Acc: 97.33%
 Epoch 37/100: Train Loss: 0.000044, Train Acc: 100.00%; Val Loss: 0.261053,
 Val Acc: 97.33%
 Epoch 38/100: Train Loss: 0.000043, Train Acc: 100.00%; Val Loss: 0.262072,
 Val Acc: 97.33%
 Epoch 39/100: Train Loss: 0.000041, Train Acc: 100.00%; Val Loss: 0.262850,
 Val Acc: 97.33%
 Epoch 40/100: Train Loss: 0.000040, Train Acc: 100.00%; Val Loss: 0.264019,
 Val Acc: 97.33%
 Epoch 41/100: Train Loss: 0.000038, Train Acc: 100.00%; Val Loss: 0.264700,
 Val Acc: 97.33%
 Epoch 42/100: Train Loss: 0.000037, Train Acc: 100.00%; Val Loss: 0.265630,
 Val Acc: 97.33%
 Epoch 43/100: Train Loss: 0.000035, Train Acc: 100.00%; Val Loss: 0.266286,
 Val Acc: 97.33%
 Epoch 44/100: Train Loss: 0.000034, Train Acc: 100.00%; Val Loss: 0.267171,
 Val Acc: 97.33%
 Epoch 45/100: Train Loss: 0.000033, Train Acc: 100.00%; Val Loss: 0.268082,
 Val Acc: 97.33%
 Epoch 46/100: Train Loss: 0.000032, Train Acc: 100.00%; Val Loss: 0.268723,
 Val Acc: 97.33%
 Epoch 47/100: Train Loss: 0.000031, Train Acc: 100.00%; Val Loss: 0.269660,
 Val Acc: 97.33%
 Epoch 48/100: Train Loss: 0.000030, Train Acc: 100.00%; Val Loss: 0.270042,
 Val Acc: 97.33%
 Epoch 49/100: Train Loss: 0.000029, Train Acc: 100.00%; Val Loss: 0.270845,
 Val Acc: 97.33%
 Epoch 50/100: Train Loss: 0.000029, Train Acc: 100.00%; Val Loss: 0.271570,
 Val Acc: 97.33%
 Epoch 51/100: Train Loss: 0.000028, Train Acc: 100.00%; Val Loss: 0.272175,
 Val Acc: 97.33%
 Epoch 52/100: Train Loss: 0.000027, Train Acc: 100.00%; Val Loss: 0.272906,
 Val Acc: 97.33%
 Epoch 53/100: Train Loss: 0.000027, Train Acc: 100.00%; Val Loss: 0.273275,
 Val Acc: 97.33%
 Epoch 54/100: Train Loss: 0.000026, Train Acc: 100.00%; Val Loss: 0.274033,

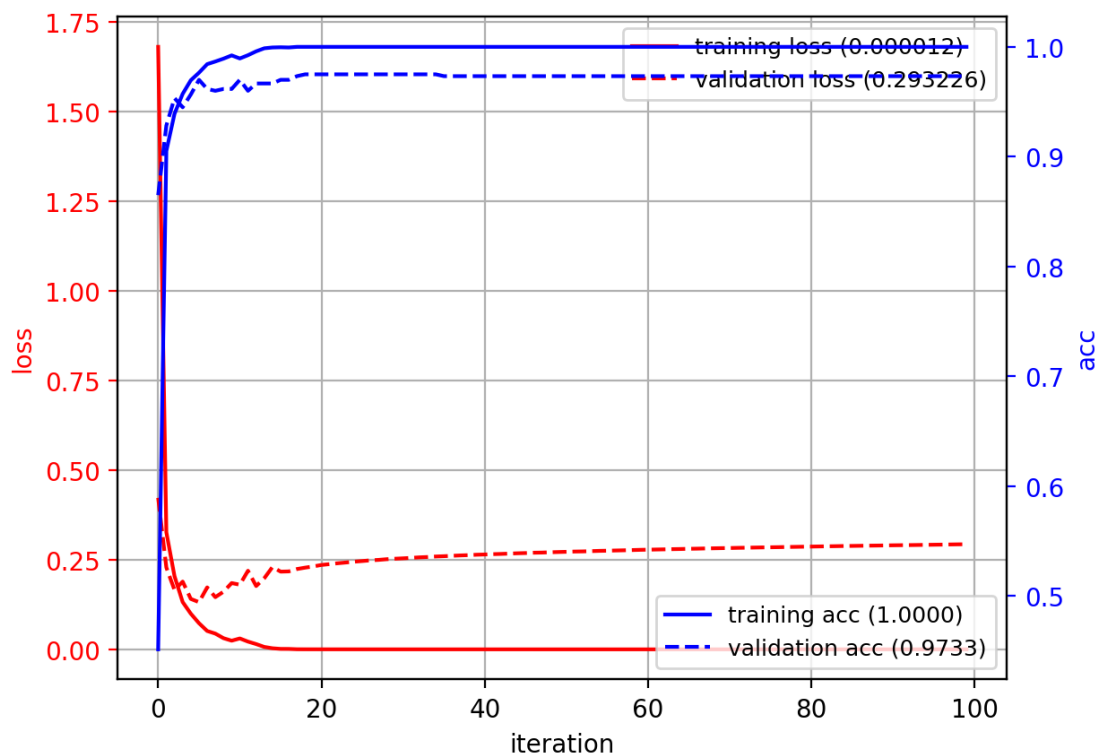
Val Acc: 97.33%
 Epoch 55/100: Train Loss: 0.000025, Train Acc: 100.00%; Val Loss: 0.274777,
 Val Acc: 97.33%
 Epoch 56/100: Train Loss: 0.000025, Train Acc: 100.00%; Val Loss: 0.275240,
 Val Acc: 97.33%
 Epoch 57/100: Train Loss: 0.000024, Train Acc: 100.00%; Val Loss: 0.275830,
 Val Acc: 97.33%
 Epoch 58/100: Train Loss: 0.000024, Train Acc: 100.00%; Val Loss: 0.276329,
 Val Acc: 97.33%
 Epoch 59/100: Train Loss: 0.000023, Train Acc: 100.00%; Val Loss: 0.276982,
 Val Acc: 97.33%
 Epoch 60/100: Train Loss: 0.000023, Train Acc: 100.00%; Val Loss: 0.277637,
 Val Acc: 97.33%
 Epoch 61/100: Train Loss: 0.000022, Train Acc: 100.00%; Val Loss: 0.278083,
 Val Acc: 97.33%
 Epoch 62/100: Train Loss: 0.000022, Train Acc: 100.00%; Val Loss: 0.278600,
 Val Acc: 97.33%
 Epoch 63/100: Train Loss: 0.000021, Train Acc: 100.00%; Val Loss: 0.279017,
 Val Acc: 97.33%
 Epoch 64/100: Train Loss: 0.000021, Train Acc: 100.00%; Val Loss: 0.279591,
 Val Acc: 97.33%
 Epoch 65/100: Train Loss: 0.000020, Train Acc: 100.00%; Val Loss: 0.279983,
 Val Acc: 97.33%
 Epoch 66/100: Train Loss: 0.000020, Train Acc: 100.00%; Val Loss: 0.280557,
 Val Acc: 97.33%
 Epoch 67/100: Train Loss: 0.000020, Train Acc: 100.00%; Val Loss: 0.280947,
 Val Acc: 97.33%
 Epoch 68/100: Train Loss: 0.000019, Train Acc: 100.00%; Val Loss: 0.281519,
 Val Acc: 97.33%
 Epoch 69/100: Train Loss: 0.000019, Train Acc: 100.00%; Val Loss: 0.281972,
 Val Acc: 97.33%
 Epoch 70/100: Train Loss: 0.000019, Train Acc: 100.00%; Val Loss: 0.282402,
 Val Acc: 97.33%
 Epoch 71/100: Train Loss: 0.000018, Train Acc: 100.00%; Val Loss: 0.282708,
 Val Acc: 97.33%
 Epoch 72/100: Train Loss: 0.000018, Train Acc: 100.00%; Val Loss: 0.283235,
 Val Acc: 97.33%
 Epoch 73/100: Train Loss: 0.000018, Train Acc: 100.00%; Val Loss: 0.283616,
 Val Acc: 97.33%
 Epoch 74/100: Train Loss: 0.000017, Train Acc: 100.00%; Val Loss: 0.284031,
 Val Acc: 97.33%
 Epoch 75/100: Train Loss: 0.000017, Train Acc: 100.00%; Val Loss: 0.284444,
 Val Acc: 97.33%
 Epoch 76/100: Train Loss: 0.000017, Train Acc: 100.00%; Val Loss: 0.284801,
 Val Acc: 97.33%
 Epoch 77/100: Train Loss: 0.000017, Train Acc: 100.00%; Val Loss: 0.285170,
 Val Acc: 97.33%
 Epoch 78/100: Train Loss: 0.000016, Train Acc: 100.00%; Val Loss: 0.285632,

Val Acc: 97.33%
 Epoch 79/100: Train Loss: 0.000016, Train Acc: 100.00%; Val Loss: 0.285987,
 Val Acc: 97.33%
 Epoch 80/100: Train Loss: 0.000016, Train Acc: 100.00%; Val Loss: 0.286391,
 Val Acc: 97.33%
 Epoch 81/100: Train Loss: 0.000016, Train Acc: 100.00%; Val Loss: 0.286778,
 Val Acc: 97.33%
 Epoch 82/100: Train Loss: 0.000015, Train Acc: 100.00%; Val Loss: 0.287178,
 Val Acc: 97.33%
 Epoch 83/100: Train Loss: 0.000015, Train Acc: 100.00%; Val Loss: 0.287559,
 Val Acc: 97.33%
 Epoch 84/100: Train Loss: 0.000015, Train Acc: 100.00%; Val Loss: 0.287917,
 Val Acc: 97.33%
 Epoch 85/100: Train Loss: 0.000015, Train Acc: 100.00%; Val Loss: 0.288349,
 Val Acc: 97.33%
 Epoch 86/100: Train Loss: 0.000014, Train Acc: 100.00%; Val Loss: 0.288621,
 Val Acc: 97.33%
 Epoch 87/100: Train Loss: 0.000014, Train Acc: 100.00%; Val Loss: 0.288982,
 Val Acc: 97.33%
 Epoch 88/100: Train Loss: 0.000014, Train Acc: 100.00%; Val Loss: 0.289312,
 Val Acc: 97.33%
 Epoch 89/100: Train Loss: 0.000014, Train Acc: 100.00%; Val Loss: 0.289607,
 Val Acc: 97.33%
 Epoch 90/100: Train Loss: 0.000014, Train Acc: 100.00%; Val Loss: 0.289948,
 Val Acc: 97.33%
 Epoch 91/100: Train Loss: 0.000013, Train Acc: 100.00%; Val Loss: 0.290380,
 Val Acc: 97.33%
 Epoch 92/100: Train Loss: 0.000013, Train Acc: 100.00%; Val Loss: 0.290643,
 Val Acc: 97.33%
 Epoch 93/100: Train Loss: 0.000013, Train Acc: 100.00%; Val Loss: 0.290906,
 Val Acc: 97.33%
 Epoch 94/100: Train Loss: 0.000013, Train Acc: 100.00%; Val Loss: 0.291222,
 Val Acc: 97.33%
 Epoch 95/100: Train Loss: 0.000013, Train Acc: 100.00%; Val Loss: 0.291558,
 Val Acc: 97.33%
 Epoch 96/100: Train Loss: 0.000013, Train Acc: 100.00%; Val Loss: 0.291940,
 Val Acc: 97.33%
 Epoch 97/100: Train Loss: 0.000012, Train Acc: 100.00%; Val Loss: 0.292237,
 Val Acc: 97.33%
 Epoch 98/100: Train Loss: 0.000012, Train Acc: 100.00%; Val Loss: 0.292660,
 Val Acc: 97.33%
 Epoch 99/100: Train Loss: 0.000012, Train Acc: 100.00%; Val Loss: 0.293024,
 Val Acc: 97.33%
 Epoch 100/100: Train Loss: 0.000012, Train Acc: 100.00%; Val Loss: 0.293226,
 Val Acc: 97.33%

8 Early Stopping

- Stopping training when the validation loss increases
 - equivalently, selecting the model/iteration with lowest validation loss.

```
[16]: # Plot the training history
plot_history(history)
```



9 Early Stopping as Regularization

- Early stopping is a form of regularization
 - limit the number of training iterations = limiting how far parameters $\mathbf{w}$ can move.
 - * e.g., if we initialize $\mathbf{w} = 0$, then early stopping limits the L2-norm of $\mathbf{w}$.
 - the amount $\mathbf{w}$ can move is adaptive to the data via the validation loss.

10 Improving Generalization with Ensembles

- **Bagging:** Bootstrap Aggregation
 - combine predictions from several models (model averaging).
- Suppose we have K models, each model has error ϵ_i
 - MSE of each model: $\text{MSE}_i = \mathbb{E}[\epsilon_i^2] = V$
- Combining the models:

- MSE of the combined model: $\text{MSE}_c = \frac{V}{K} + \frac{K-1}{K}C$
 - * $C = \mathbb{E}[\epsilon_i \epsilon_j]$ is the correlation between errors of the i-th and j-th models.
- If the errors are uncorrelated ($C = 0$), then the MSE decreases!
 - $\text{MSE}_c = \frac{V}{K}$

11 Bagging for NN

- Typically bagging needs to create a new dataset by sampling with replacement of the original dataset, and then learning a new model on each generated dataset.
- For NN, there are enough solution points
 - we can use the same dataset, and different initializations (or change hyperparameters).
 - this should give models with partially uncorrelated errors.
- This is a **very** popular way to boost performance.

```
[17]: # Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True

allnn = []
allhistory = []
for bag in range(5):

    model = CNNWithL2()

    # Move model to device (GPU if available)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)
    print(f"Using device: {device}")

    optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9,
    ↪nesterov=True)
    history, model = train_model(model, train_loader, valid_loader, criterion,
    ↪optimizer, epochs=100)

    allnn.append(model)
    allhistory.append(history)
```

Using device: cuda

Epoch 1/100: Train Loss: 1.620371, Train Acc: 44.61%; Val Loss: 0.468100,
Val Acc: 84.33%

Epoch 2/100: Train Loss: 0.329343, Train Acc: 90.11%; Val Loss: 0.245200,
Val Acc: 93.17%

Epoch 3/100: Train Loss: 0.203770, Train Acc: 93.59%; Val Loss: 0.214860,

Val Acc: 93.00%
 EarlyStopping counter: 1 out of 5
 Epoch 4/100: Train Loss: 0.133710, Train Acc: 95.72%; Val Loss: 0.158682, Val Acc: 95.17%
 Epoch 5/100: Train Loss: 0.095974, Train Acc: 96.89%; Val Loss: 0.199730, Val Acc: 94.50%
 EarlyStopping counter: 1 out of 5
 Epoch 6/100: Train Loss: 0.082481, Train Acc: 97.37%; Val Loss: 0.145677, Val Acc: 95.83%
 Epoch 7/100: Train Loss: 0.060736, Train Acc: 98.00%; Val Loss: 0.149765, Val Acc: 95.83%
 EarlyStopping counter: 1 out of 5
 Epoch 8/100: Train Loss: 0.040754, Train Acc: 98.78%; Val Loss: 0.132005, Val Acc: 96.17%
 Epoch 9/100: Train Loss: 0.032565, Train Acc: 98.91%; Val Loss: 0.176460, Val Acc: 95.50%
 EarlyStopping counter: 1 out of 5
 Epoch 10/100: Train Loss: 0.032279, Train Acc: 99.04%; Val Loss: 0.238308, Val Acc: 96.00%
 EarlyStopping counter: 2 out of 5
 Epoch 11/100: Train Loss: 0.029881, Train Acc: 99.20%; Val Loss: 0.176865, Val Acc: 96.50%
 Epoch 12/100: Train Loss: 0.017214, Train Acc: 99.44%; Val Loss: 0.183611, Val Acc: 96.50%
 EarlyStopping counter: 1 out of 5
 Epoch 13/100: Train Loss: 0.021913, Train Acc: 99.41%; Val Loss: 0.156942, Val Acc: 96.50%
 EarlyStopping counter: 2 out of 5
 Epoch 14/100: Train Loss: 0.012814, Train Acc: 99.61%; Val Loss: 0.156906, Val Acc: 96.33%
 EarlyStopping counter: 3 out of 5
 Epoch 15/100: Train Loss: 0.011452, Train Acc: 99.69%; Val Loss: 0.179287, Val Acc: 97.00%
 Epoch 16/100: Train Loss: 0.020812, Train Acc: 99.43%; Val Loss: 0.173752, Val Acc: 96.67%
 EarlyStopping counter: 1 out of 5
 Epoch 17/100: Train Loss: 0.012141, Train Acc: 99.70%; Val Loss: 0.162341, Val Acc: 96.50%
 EarlyStopping counter: 2 out of 5
 Epoch 18/100: Train Loss: 0.003358, Train Acc: 99.98%; Val Loss: 0.152713, Val Acc: 97.33%
 Epoch 19/100: Train Loss: 0.001990, Train Acc: 100.00%; Val Loss: 0.163681, Val Acc: 97.17%
 EarlyStopping counter: 1 out of 5
 Epoch 20/100: Train Loss: 0.001826, Train Acc: 100.00%; Val Loss: 0.164104, Val Acc: 97.17%
 EarlyStopping counter: 2 out of 5
 Epoch 21/100: Train Loss: 0.001752, Train Acc: 100.00%; Val Loss: 0.166261,

Val Acc: 97.17%
 EarlyStopping counter: 3 out of 5
 Epoch 22/100: Train Loss: 0.001722, Train Acc: 100.00%; Val Loss: 0.169351, Val Acc: 97.17%
 EarlyStopping counter: 4 out of 5
 Epoch 23/100: Train Loss: 0.001702, Train Acc: 100.00%; Val Loss: 0.171411, Val Acc: 97.17%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered
 Using device: cuda
 Epoch 1/100: Train Loss: 1.642271, Train Acc: 44.09%; Val Loss: 0.444070, Val Acc: 86.33%
 Epoch 2/100: Train Loss: 0.329157, Train Acc: 90.04%; Val Loss: 0.201360, Val Acc: 93.67%
 Epoch 3/100: Train Loss: 0.200654, Train Acc: 94.09%; Val Loss: 0.192785, Val Acc: 94.00%
 Epoch 4/100: Train Loss: 0.132160, Train Acc: 96.06%; Val Loss: 0.171013, Val Acc: 94.17%
 Epoch 5/100: Train Loss: 0.099796, Train Acc: 96.89%; Val Loss: 0.144368, Val Acc: 94.83%
 Epoch 6/100: Train Loss: 0.074152, Train Acc: 97.57%; Val Loss: 0.175120, Val Acc: 94.67%
 EarlyStopping counter: 1 out of 5
 Epoch 7/100: Train Loss: 0.065484, Train Acc: 97.78%; Val Loss: 0.149223, Val Acc: 96.50%
 Epoch 8/100: Train Loss: 0.048841, Train Acc: 98.28%; Val Loss: 0.118805, Val Acc: 97.17%
 Epoch 9/100: Train Loss: 0.034310, Train Acc: 98.93%; Val Loss: 0.156861, Val Acc: 96.67%
 EarlyStopping counter: 1 out of 5
 Epoch 10/100: Train Loss: 0.034876, Train Acc: 99.00%; Val Loss: 0.148262, Val Acc: 96.00%
 EarlyStopping counter: 2 out of 5
 Epoch 11/100: Train Loss: 0.017695, Train Acc: 99.46%; Val Loss: 0.166790, Val Acc: 96.00%
 EarlyStopping counter: 3 out of 5
 Epoch 12/100: Train Loss: 0.018290, Train Acc: 99.37%; Val Loss: 0.181502, Val Acc: 95.17%
 EarlyStopping counter: 4 out of 5
 Epoch 13/100: Train Loss: 0.021440, Train Acc: 99.41%; Val Loss: 0.168426, Val Acc: 96.17%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered
 Using device: cuda
 Epoch 1/100: Train Loss: 1.636279, Train Acc: 41.50%; Val Loss: 0.425871, Val Acc: 86.83%
 Epoch 2/100: Train Loss: 0.338937, Train Acc: 89.98%; Val Loss: 0.271695, Val Acc: 93.17%

Epoch 3/100: Train Loss: 0.204298, Train Acc: 93.65%; Val Loss: 0.197697, Val Acc: 94.17%
 Epoch 4/100: Train Loss: 0.140447, Train Acc: 95.59%; Val Loss: 0.203720, Val Acc: 94.67%
 Epoch 5/100: Train Loss: 0.095893, Train Acc: 96.83%; Val Loss: 0.193911, Val Acc: 94.83%
 Epoch 6/100: Train Loss: 0.074691, Train Acc: 97.46%; Val Loss: 0.200921, Val Acc: 94.83%
 EarlyStopping counter: 1 out of 5
 Epoch 7/100: Train Loss: 0.064152, Train Acc: 97.94%; Val Loss: 0.175583, Val Acc: 96.00%
 Epoch 8/100: Train Loss: 0.046281, Train Acc: 98.56%; Val Loss: 0.209746, Val Acc: 95.50%
 EarlyStopping counter: 1 out of 5
 Epoch 9/100: Train Loss: 0.036268, Train Acc: 98.83%; Val Loss: 0.149686, Val Acc: 96.83%
 Epoch 10/100: Train Loss: 0.027809, Train Acc: 99.07%; Val Loss: 0.181953, Val Acc: 96.00%
 EarlyStopping counter: 1 out of 5
 Epoch 11/100: Train Loss: 0.014475, Train Acc: 99.61%; Val Loss: 0.167803, Val Acc: 96.33%
 EarlyStopping counter: 2 out of 5
 Epoch 12/100: Train Loss: 0.013429, Train Acc: 99.57%; Val Loss: 0.200195, Val Acc: 96.33%
 EarlyStopping counter: 3 out of 5
 Epoch 13/100: Train Loss: 0.013588, Train Acc: 99.70%; Val Loss: 0.212116, Val Acc: 96.50%
 EarlyStopping counter: 4 out of 5
 Epoch 14/100: Train Loss: 0.008747, Train Acc: 99.83%; Val Loss: 0.219695, Val Acc: 96.67%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered
 Using device: cuda
 Epoch 1/100: Train Loss: 1.838033, Train Acc: 39.78%; Val Loss: 0.507780, Val Acc: 84.67%
 Epoch 2/100: Train Loss: 0.348409, Train Acc: 89.87%; Val Loss: 0.211512, Val Acc: 93.00%
 Epoch 3/100: Train Loss: 0.205593, Train Acc: 93.63%; Val Loss: 0.185261, Val Acc: 94.67%
 Epoch 4/100: Train Loss: 0.139137, Train Acc: 95.96%; Val Loss: 0.177526, Val Acc: 94.50%
 EarlyStopping counter: 1 out of 5
 Epoch 5/100: Train Loss: 0.100503, Train Acc: 97.06%; Val Loss: 0.175824, Val Acc: 95.17%
 Epoch 6/100: Train Loss: 0.080893, Train Acc: 97.43%; Val Loss: 0.152271, Val Acc: 96.17%
 Epoch 7/100: Train Loss: 0.052190, Train Acc: 98.31%; Val Loss: 0.167834, Val Acc: 96.33%

Epoch 8/100: Train Loss: 0.048136, Train Acc: 98.65%; Val Loss: 0.169361,
 Val Acc: 95.83%
 EarlyStopping counter: 1 out of 5
 Epoch 9/100: Train Loss: 0.039541, Train Acc: 98.80%; Val Loss: 0.178365,
 Val Acc: 95.67%
 EarlyStopping counter: 2 out of 5
 Epoch 10/100: Train Loss: 0.021785, Train Acc: 99.37%; Val Loss: 0.223549,
 Val Acc: 95.67%
 EarlyStopping counter: 3 out of 5
 Epoch 11/100: Train Loss: 0.022647, Train Acc: 99.31%; Val Loss: 0.195512,
 Val Acc: 96.00%
 EarlyStopping counter: 4 out of 5
 Epoch 12/100: Train Loss: 0.021707, Train Acc: 99.39%; Val Loss: 0.206068,
 Val Acc: 96.00%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered
 Using device: cuda
 Epoch 1/100: Train Loss: 1.584540, Train Acc: 45.80%; Val Loss: 0.441123,
 Val Acc: 86.67%
 Epoch 2/100: Train Loss: 0.343413, Train Acc: 89.35%; Val Loss: 0.242832,
 Val Acc: 92.00%
 Epoch 3/100: Train Loss: 0.203329, Train Acc: 93.50%; Val Loss: 0.197196,
 Val Acc: 93.17%
 Epoch 4/100: Train Loss: 0.144302, Train Acc: 95.59%; Val Loss: 0.180510,
 Val Acc: 94.83%
 Epoch 5/100: Train Loss: 0.097791, Train Acc: 96.67%; Val Loss: 0.233445,
 Val Acc: 94.67%
 EarlyStopping counter: 1 out of 5
 Epoch 6/100: Train Loss: 0.071802, Train Acc: 97.85%; Val Loss: 0.177414,
 Val Acc: 96.17%
 Epoch 7/100: Train Loss: 0.058626, Train Acc: 98.04%; Val Loss: 0.158420,
 Val Acc: 95.50%
 EarlyStopping counter: 1 out of 5
 Epoch 8/100: Train Loss: 0.048679, Train Acc: 98.54%; Val Loss: 0.208895,
 Val Acc: 95.00%
 EarlyStopping counter: 2 out of 5
 Epoch 9/100: Train Loss: 0.033622, Train Acc: 98.89%; Val Loss: 0.204051,
 Val Acc: 95.33%
 EarlyStopping counter: 3 out of 5
 Epoch 10/100: Train Loss: 0.034087, Train Acc: 98.98%; Val Loss: 0.214439,
 Val Acc: 96.33%
 Epoch 11/100: Train Loss: 0.027750, Train Acc: 99.13%; Val Loss: 0.176449,
 Val Acc: 96.17%
 EarlyStopping counter: 1 out of 5
 Epoch 12/100: Train Loss: 0.021090, Train Acc: 99.35%; Val Loss: 0.212935,
 Val Acc: 97.00%
 Epoch 13/100: Train Loss: 0.017898, Train Acc: 99.44%; Val Loss: 0.242227,
 Val Acc: 95.83%

```

EarlyStopping counter: 1 out of 5
Epoch 14/100:   Train Loss: 0.025401, Train Acc: 99.30%;   Val Loss: 0.218800,
Val Acc: 96.00%
EarlyStopping counter: 2 out of 5
Epoch 15/100:   Train Loss: 0.013955, Train Acc: 99.65%;   Val Loss: 0.183772,
Val Acc: 97.00%
EarlyStopping counter: 3 out of 5
Epoch 16/100:   Train Loss: 0.008135, Train Acc: 99.78%;   Val Loss: 0.277358,
Val Acc: 95.17%
EarlyStopping counter: 4 out of 5
Epoch 17/100:   Train Loss: 0.012128, Train Acc: 99.69%;   Val Loss: 0.277929,
Val Acc: 95.33%
EarlyStopping counter: 5 out of 5
Early stopping triggered

```

12 Example: Ensembles

- train 5 CNNs with same architectures but different initializations

```

[18]: import pdb
def ensemble_prediction(models, eval_loader):
    num_models = len(models)
    eval_correct = {i:0 for i in range(num_models+1)}
    eval_total = {i:0 for i in range(num_models+1)}
    allpred = {i:[] for i in range(num_models+1)}

    with torch.no_grad():
        for data, target in eval_loader:
            data = data.to(device)
            outputs = []
            for i,model in enumerate(models):
                model.eval()
                output = model(data)
                _, predicted = torch.max(output.data, 1)
                predicted = predicted.cpu()
                eval_total[i] += target.size(0)
                eval_correct[i] += (predicted == target).sum().item()
                allpred[i].append(predicted.numpy())
                outputs.append(output.cpu())
            eval_total[num_models] += target.size(0)
            _, predicted = torch.max(torch.stack(outputs,dim=-1).sum(dim=-1).
↪data, 1)
            allpred[num_models].append(predicted.numpy())
            eval_correct[num_models] += (predicted == target).sum().item()

    eval_accuracy = {i:100 * eval_correct[i] / eval_total[i] for i in_
↪range(num_models+1)}

```

```
return eval_accuracy, allpred
```

```
[19]: eval_accuracy, allpred = ensemble_prediction(allnn, test_loader)
```

- various errors for the individual NNs
- ensembling the predictions improves accuracy

```
[20]: # compute error for individual models
for i, acc in eval_accuracy.items():
    if i < len(allnn):
        print("model {} test accuracy: {}".format(i, acc))

# compute error for ensemble
print("combined model test accuracy: {}".format(eval_accuracy[len(allnn)]))
```

```
model 0 test accuracy: 96.58
model 1 test accuracy: 95.81
model 2 test accuracy: 96.18
model 3 test accuracy: 96.12
model 4 test accuracy: 95.89
combined model test accuracy: 96.65
```

13 Dropout

- **Problem:** training a large number of NNs for an ensemble is time consuming.
- **Goal:** approximate a really large ensemble of models
 - create an ensemble by randomly removing an input/hidden node with some probability.
- number of models is exponential in the number of nodes.
- share the same parameters for each sub-model.
 - we can train all models in the ensemble at the same time.

14 Dropout Training

- During training, randomly “drop out” each node with probability p
 - a dropped-out node is not used for calculating the prediction or weight updating.
 - trains a reduced network in each iteration.
- During test time, use all the nodes for prediction and scale output by $1 - p$.
 - this keeps the expected value of the node to be the same as during training.
 - analogous to computing predictions from all models, and then averages the predictions.
- Dropout is implemented as a layer.
- Example:
 - apply dropout layer after last feature layer and 1st dense layer.

```
[21]: class CNNWithDropout(CNN):
        def __init__(self):
```



```

    super(CNNWithDropout, self).__init__()
    self.drop1d = nn.Dropout(p=0.5)
    self.drop2d = nn.Dropout2d(p=0.5)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.relu(self.conv3(x))
        x = self.drop2d(x)
        x = x.view(x.size(0), -1)  # Flatten
        x = self.relu(self.fc1(x))
        x = self.fc2(self.drop1d(x))
        return x

```

```

[22]: model = CNNWithDropout()

# Print model architecture
print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
                               optimizer, epochs=100, writer='runs/dropout')

# Plot the training history
plot_history(history)

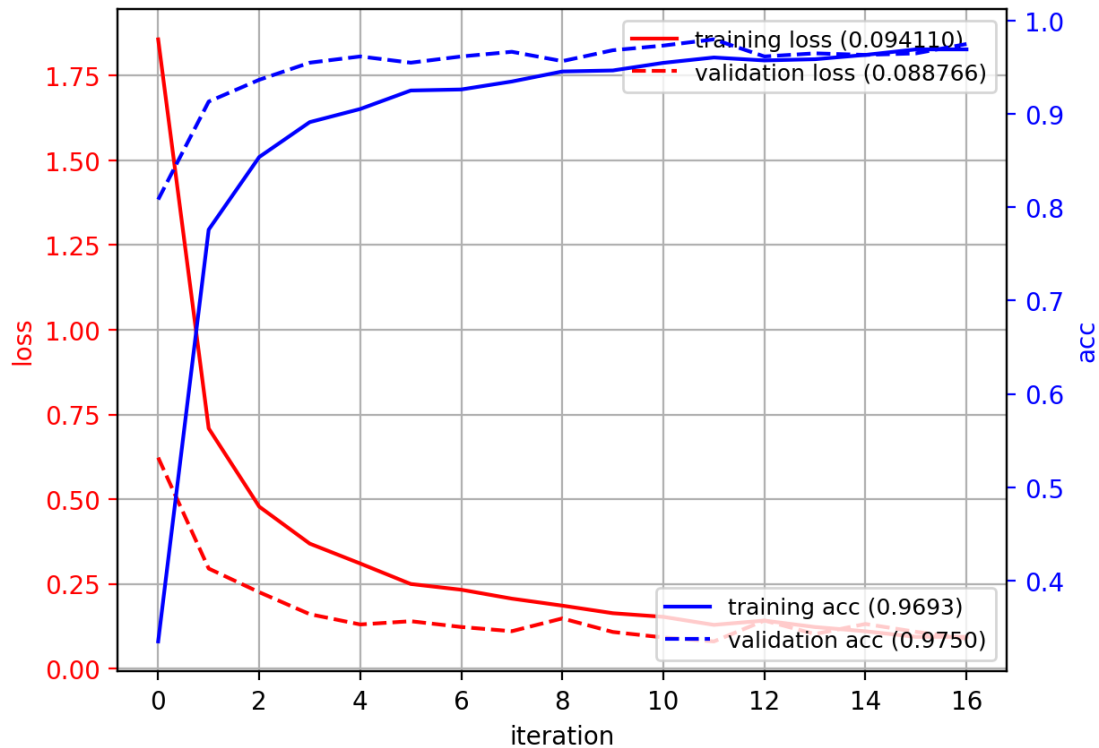
```

```

CNNWithDropout(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
  (drop1d): Dropout(p=0.5, inplace=False)
  (drop2d): Dropout2d(p=0.5, inplace=False)
)
Using device: cuda
Epoch 1/100:   Train Loss: 1.856782, Train Acc: 33.48%;   Val Loss: 0.622844,
Val Acc: 80.83%

```

Epoch 2/100: Train Loss: 0.708875, Train Acc: 77.61%; Val Loss: 0.295382, Val Acc: 91.33%
 Epoch 3/100: Train Loss: 0.478177, Train Acc: 85.39%; Val Loss: 0.225654, Val Acc: 93.67%
 Epoch 4/100: Train Loss: 0.368947, Train Acc: 89.13%; Val Loss: 0.160814, Val Acc: 95.50%
 Epoch 5/100: Train Loss: 0.310231, Train Acc: 90.54%; Val Loss: 0.130448, Val Acc: 96.17%
 Epoch 6/100: Train Loss: 0.249824, Train Acc: 92.52%; Val Loss: 0.140031, Val Acc: 95.50%
 EarlyStopping counter: 1 out of 5
 Epoch 7/100: Train Loss: 0.232903, Train Acc: 92.63%; Val Loss: 0.122971, Val Acc: 96.17%
 EarlyStopping counter: 2 out of 5
 Epoch 8/100: Train Loss: 0.206619, Train Acc: 93.48%; Val Loss: 0.110569, Val Acc: 96.67%
 Epoch 9/100: Train Loss: 0.185993, Train Acc: 94.56%; Val Loss: 0.148278, Val Acc: 95.67%
 EarlyStopping counter: 1 out of 5
 Epoch 10/100: Train Loss: 0.163651, Train Acc: 94.67%; Val Loss: 0.108261, Val Acc: 96.83%
 Epoch 11/100: Train Loss: 0.152881, Train Acc: 95.48%; Val Loss: 0.092072, Val Acc: 97.33%
 Epoch 12/100: Train Loss: 0.129267, Train Acc: 96.06%; Val Loss: 0.080383, Val Acc: 98.00%
 Epoch 13/100: Train Loss: 0.141842, Train Acc: 95.74%; Val Loss: 0.142028, Val Acc: 96.17%
 EarlyStopping counter: 1 out of 5
 Epoch 14/100: Train Loss: 0.122863, Train Acc: 95.87%; Val Loss: 0.102299, Val Acc: 96.50%
 EarlyStopping counter: 2 out of 5
 Epoch 15/100: Train Loss: 0.110372, Train Acc: 96.33%; Val Loss: 0.131860, Val Acc: 96.33%
 EarlyStopping counter: 3 out of 5
 Epoch 16/100: Train Loss: 0.093787, Train Acc: 96.93%; Val Loss: 0.108714, Val Acc: 96.50%
 EarlyStopping counter: 4 out of 5
 Epoch 17/100: Train Loss: 0.094110, Train Acc: 96.93%; Val Loss: 0.088766, Val Acc: 97.50%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered



- accuracy improves

```
[23]: acc = evaluate_model(model, test_loader)
      print("test accuracy:", acc)
```

test accuracy: 96.77

15 Effects of Dropout

- dropping a node means the node's output is 0
 - i.e., the gradient is 0, which blocks some paths when computing gradients.
 - helps with vanishing gradient problem
- model averaging reduces the error.
- Dropout *reduces* the effective capacity of the network.
 - may need to increase the model size.
- At any time a node h will be removed, i.e., that feature is missing.
 - then the other features must be sufficient to solve the classification problem
 - classifier doesn't depend exclusively on one feature.
 - increases classifier robustness.

16 Data augmentation

- Problem: not enough data

- Solution: artificially permute the data to increase the dataset size
 - goal: make the network invariant to the permutations
 - examples: translate image, flip image, add pixel noise, rotate image, deform image, etc.

```
[24]: # build the data augmenter

transform_train = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomRotation(degrees=10),           #┐
    ↪rotation_range=10
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)), #┐
    ↪width_shift_range=0.1, height_shift_range=0.1
    transforms.RandomAffine(degrees=0, shear=10),      # shear_range=0.1
    transforms.RandomResizedCrop(28, scale=(0.9, 1.1)), # zoom_range=0.1
    transforms.ToTensor()
])

class MyDataset(Dataset):
    def __init__(self, imgs, labels, transform=None):
        super(MyDataset, self).__init__()
        self.transform = transform
        self.imgs = imgs
        self.labels = labels

    def __getitem__(self, item):
        img = self.transform(self.imgs[item])
        target = self.labels[item]
        return img, target

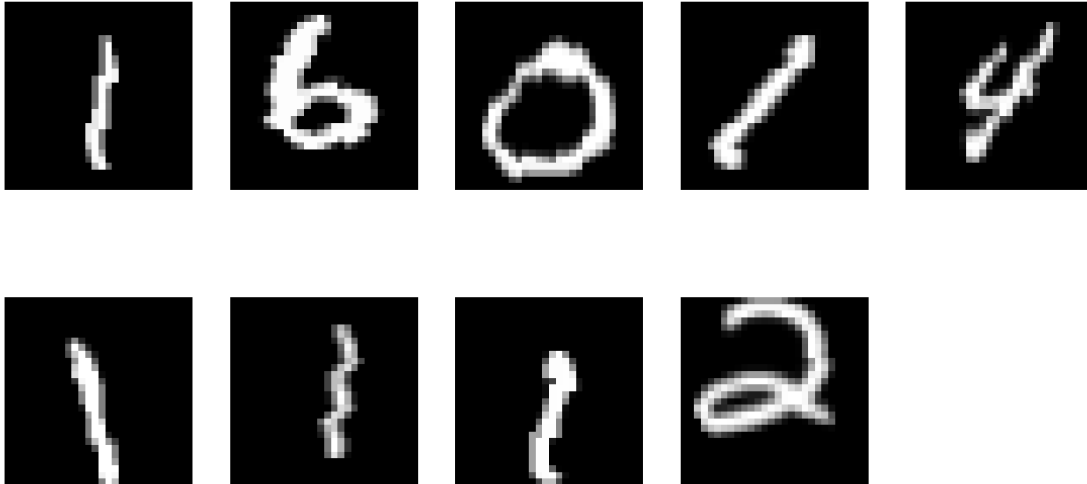
    def __len__(self):
        return len(self.labels)
```

```
[25]: train_aug_dataset = MyDataset(vtrainI,vtrainY,transform_train)
imgs = []

cnt = 0
for augx, y in train_aug_dataset:
    imgs.append(augx[0].numpy())
    cnt += 1
    if cnt > 8:
        # we need to break the loop by hand because
        # the generator loops indefinitely
        break
```

- Example of original (top-left) and augmented data

```
[26]: plt.figure(figsize=(8,4))
show_imgs(imgs, nc=5)
plt.show()
```



- Any transformation is okay, as long as the transformation doesn't confuse the classes.
 - e.g. don't rotate "6" so that it looks like a "9"
- Data augmentation improves robustness or invariance to selected transformations
 - encoded in the parameters of the model.

```
[27]: model = CNNWithDropout()

# Print model architecture
print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

train_aug_loader = DataLoader(train_aug_dataset, batch_size=batch_size,
                               shuffle=True)

# Train the model
history, model = train_model(model, train_aug_loader, valid_loader, criterion,
                               optimizer, epochs=100, writer='runs/daug')

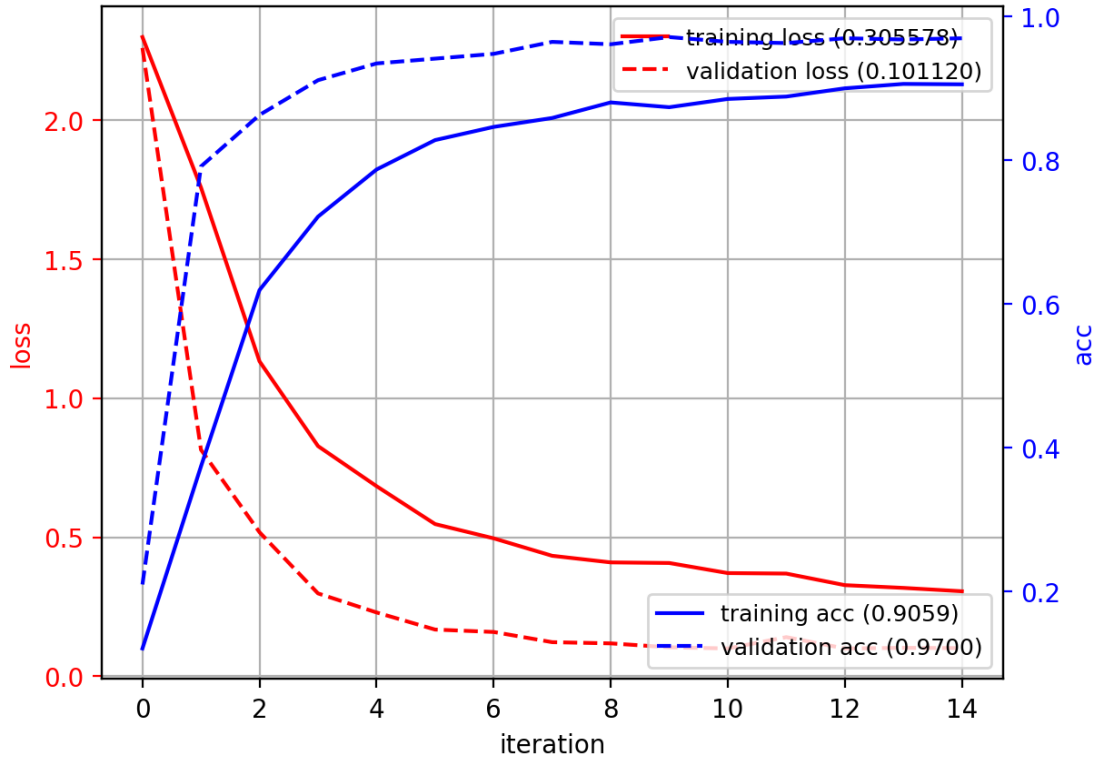
# Plot the training history
plot_history(history)
```

```

CNNWithDropout(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
  (drop1d): Dropout(p=0.5, inplace=False)
  (drop2d): Dropout2d(p=0.5, inplace=False)
)
Using device: cuda
Epoch 1/100:   Train Loss: 2.297832, Train Acc: 12.09%;   Val Loss: 2.258782,
Val Acc: 21.00%
Epoch 2/100:   Train Loss: 1.757125, Train Acc: 37.43%;   Val Loss: 0.814421,
Val Acc: 79.17%
Epoch 3/100:   Train Loss: 1.132176, Train Acc: 61.96%;   Val Loss: 0.517821,
Val Acc: 86.33%
Epoch 4/100:   Train Loss: 0.827414, Train Acc: 72.19%;   Val Loss: 0.297796,
Val Acc: 91.17%
Epoch 5/100:   Train Loss: 0.683318, Train Acc: 78.74%;   Val Loss: 0.229370,
Val Acc: 93.50%
Epoch 6/100:   Train Loss: 0.547148, Train Acc: 82.83%;   Val Loss: 0.167907,
Val Acc: 94.17%
Epoch 7/100:   Train Loss: 0.495886, Train Acc: 84.67%;   Val Loss: 0.159274,
Val Acc: 94.83%
Epoch 8/100:   Train Loss: 0.433135, Train Acc: 85.91%;   Val Loss: 0.122584,
Val Acc: 96.50%
Epoch 9/100:   Train Loss: 0.409367, Train Acc: 88.07%;   Val Loss: 0.118237,
Val Acc: 96.17%
EarlyStopping counter: 1 out of 5
Epoch 10/100:  Train Loss: 0.407214, Train Acc: 87.41%;   Val Loss: 0.104176,
Val Acc: 97.17%
Epoch 11/100:  Train Loss: 0.370984, Train Acc: 88.56%;   Val Loss: 0.099936,
Val Acc: 96.50%
EarlyStopping counter: 1 out of 5
Epoch 12/100:  Train Loss: 0.369157, Train Acc: 88.89%;   Val Loss: 0.140652,
Val Acc: 96.33%
EarlyStopping counter: 2 out of 5
Epoch 13/100:  Train Loss: 0.327608, Train Acc: 90.04%;   Val Loss: 0.099902,
Val Acc: 97.00%
EarlyStopping counter: 3 out of 5
Epoch 14/100:  Train Loss: 0.317724, Train Acc: 90.65%;   Val Loss: 0.102173,
Val Acc: 96.83%
EarlyStopping counter: 4 out of 5
Epoch 15/100:  Train Loss: 0.305578, Train Acc: 90.59%;   Val Loss: 0.101120,
Val Acc: 97.00%
EarlyStopping counter: 5 out of 5

```

Early stopping triggered



- Model Training
 - passes data through the customized dataset first
 - runs dataloader as data generator and fit in parallel
- the dataset changes each epoch, so validation error will change a lot.
 - we disable early stopping and just let it run for 100 epochs.
- use TensorBoard to view the current results in real-time.
 - Create a **TensorBoard** object and use it as the callback.
 - This will write the history information to the logs directory.

```
[28]: model = CNNWithDropout()

# Print model architecture
print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)
```

```

# Train the model
history, model = train_model(model, train_aug_loader, valid_loader, criterion,
    ↪optimizer, epochs=100, patience=-1, writer='runs/daug-100')

# Plot the training history
plot_history(history)

```

```

CNNWithDropout(
    (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
    (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
    (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (fc1): Linear(in_features=3920, out_features=50, bias=True)
    (fc2): Linear(in_features=50, out_features=10, bias=True)
    (relu): ReLU()
    (softmax): Softmax(dim=1)
    (drop1d): Dropout(p=0.5, inplace=False)
    (drop2d): Dropout2d(p=0.5, inplace=False)
)
Using device: cuda
Epoch 1/100:   Train Loss: 2.175980, Train Acc: 20.35%;   Val Loss: 1.243000,
Val Acc: 62.17%
Epoch 2/100:   Train Loss: 1.339405, Train Acc: 54.20%;   Val Loss: 0.482699,
Val Acc: 87.17%
Epoch 3/100:   Train Loss: 0.907097, Train Acc: 69.56%;   Val Loss: 0.293861,
Val Acc: 90.33%
Epoch 4/100:   Train Loss: 0.681126, Train Acc: 78.33%;   Val Loss: 0.218618,
Val Acc: 94.00%
Epoch 5/100:   Train Loss: 0.594205, Train Acc: 81.57%;   Val Loss: 0.194515,
Val Acc: 95.00%
Epoch 6/100:   Train Loss: 0.506758, Train Acc: 84.37%;   Val Loss: 0.218005,
Val Acc: 93.50%
Epoch 7/100:   Train Loss: 0.475873, Train Acc: 85.30%;   Val Loss: 0.137679,
Val Acc: 95.83%
Epoch 8/100:   Train Loss: 0.430564, Train Acc: 87.20%;   Val Loss: 0.159742,
Val Acc: 95.00%
Epoch 9/100:   Train Loss: 0.399365, Train Acc: 88.09%;   Val Loss: 0.147542,
Val Acc: 94.67%
Epoch 10/100:  Train Loss: 0.357367, Train Acc: 89.02%;   Val Loss: 0.128558,
Val Acc: 95.33%
Epoch 11/100:  Train Loss: 0.355269, Train Acc: 89.39%;   Val Loss: 0.144938,
Val Acc: 96.17%
Epoch 12/100:  Train Loss: 0.330639, Train Acc: 90.19%;   Val Loss: 0.116257,
Val Acc: 96.50%
Epoch 13/100:  Train Loss: 0.306665, Train Acc: 90.98%;   Val Loss: 0.113742,
Val Acc: 96.67%
Epoch 14/100:  Train Loss: 0.290908, Train Acc: 91.28%;   Val Loss: 0.096498,
Val Acc: 96.83%

```


Epoch 15/100: Train Loss: 0.281811, Train Acc: 91.59%; Val Loss: 0.111831, Val Acc: 96.83%

Epoch 16/100: Train Loss: 0.282509, Train Acc: 91.20%; Val Loss: 0.103017, Val Acc: 96.50%

Epoch 17/100: Train Loss: 0.246989, Train Acc: 92.28%; Val Loss: 0.101214, Val Acc: 97.67%

Epoch 18/100: Train Loss: 0.268398, Train Acc: 92.22%; Val Loss: 0.115013, Val Acc: 96.17%

Epoch 19/100: Train Loss: 0.256693, Train Acc: 92.48%; Val Loss: 0.110682, Val Acc: 97.00%

Epoch 20/100: Train Loss: 0.248818, Train Acc: 92.52%; Val Loss: 0.110212, Val Acc: 96.83%

Epoch 21/100: Train Loss: 0.237679, Train Acc: 92.89%; Val Loss: 0.108385, Val Acc: 96.67%

Epoch 22/100: Train Loss: 0.244866, Train Acc: 92.67%; Val Loss: 0.127022, Val Acc: 96.67%

Epoch 23/100: Train Loss: 0.234243, Train Acc: 93.48%; Val Loss: 0.106847, Val Acc: 97.33%

Epoch 24/100: Train Loss: 0.249280, Train Acc: 93.00%; Val Loss: 0.117176, Val Acc: 97.33%

Epoch 25/100: Train Loss: 0.220503, Train Acc: 93.31%; Val Loss: 0.070500, Val Acc: 97.67%

Epoch 26/100: Train Loss: 0.229948, Train Acc: 93.28%; Val Loss: 0.083749, Val Acc: 97.50%

Epoch 27/100: Train Loss: 0.223983, Train Acc: 93.28%; Val Loss: 0.112642, Val Acc: 97.00%

Epoch 28/100: Train Loss: 0.212207, Train Acc: 93.80%; Val Loss: 0.099342, Val Acc: 97.00%

Epoch 29/100: Train Loss: 0.218289, Train Acc: 93.30%; Val Loss: 0.084248, Val Acc: 97.83%

Epoch 30/100: Train Loss: 0.201090, Train Acc: 93.80%; Val Loss: 0.076514, Val Acc: 98.33%

Epoch 31/100: Train Loss: 0.210137, Train Acc: 93.67%; Val Loss: 0.075768, Val Acc: 97.67%

Epoch 32/100: Train Loss: 0.210481, Train Acc: 94.00%; Val Loss: 0.077467, Val Acc: 97.67%

Epoch 33/100: Train Loss: 0.195491, Train Acc: 94.46%; Val Loss: 0.084374, Val Acc: 97.83%

Epoch 34/100: Train Loss: 0.193401, Train Acc: 94.11%; Val Loss: 0.065861, Val Acc: 98.33%

Epoch 35/100: Train Loss: 0.189430, Train Acc: 94.31%; Val Loss: 0.074475, Val Acc: 97.83%

Epoch 36/100: Train Loss: 0.187922, Train Acc: 94.41%; Val Loss: 0.075829, Val Acc: 97.67%

Epoch 37/100: Train Loss: 0.196728, Train Acc: 94.33%; Val Loss: 0.074402, Val Acc: 97.83%

Epoch 38/100: Train Loss: 0.213607, Train Acc: 93.56%; Val Loss: 0.081971, Val Acc: 98.17%

Epoch 39/100: Train Loss: 0.187246, Train Acc: 94.65%; Val Loss: 0.070367, Val Acc: 98.17%

Epoch 40/100: Train Loss: 0.196170, Train Acc: 93.83%; Val Loss: 0.090454, Val Acc: 97.17%

Epoch 41/100: Train Loss: 0.185668, Train Acc: 94.24%; Val Loss: 0.070677, Val Acc: 97.33%

Epoch 42/100: Train Loss: 0.190212, Train Acc: 94.02%; Val Loss: 0.080926, Val Acc: 97.67%

Epoch 43/100: Train Loss: 0.188539, Train Acc: 94.52%; Val Loss: 0.069338, Val Acc: 98.00%

Epoch 44/100: Train Loss: 0.180890, Train Acc: 94.50%; Val Loss: 0.076412, Val Acc: 98.00%

Epoch 45/100: Train Loss: 0.175821, Train Acc: 94.85%; Val Loss: 0.062882, Val Acc: 97.83%

Epoch 46/100: Train Loss: 0.181656, Train Acc: 94.46%; Val Loss: 0.071188, Val Acc: 98.00%

Epoch 47/100: Train Loss: 0.188648, Train Acc: 94.78%; Val Loss: 0.085658, Val Acc: 98.00%

Epoch 48/100: Train Loss: 0.160992, Train Acc: 94.76%; Val Loss: 0.084718, Val Acc: 98.00%

Epoch 49/100: Train Loss: 0.170842, Train Acc: 95.24%; Val Loss: 0.061813, Val Acc: 98.00%

Epoch 50/100: Train Loss: 0.171351, Train Acc: 95.00%; Val Loss: 0.077609, Val Acc: 98.00%

Epoch 51/100: Train Loss: 0.159608, Train Acc: 94.67%; Val Loss: 0.065036, Val Acc: 98.00%

Epoch 52/100: Train Loss: 0.165521, Train Acc: 94.70%; Val Loss: 0.062655, Val Acc: 98.33%

Epoch 53/100: Train Loss: 0.159678, Train Acc: 95.33%; Val Loss: 0.074378, Val Acc: 97.33%

Epoch 54/100: Train Loss: 0.172373, Train Acc: 94.98%; Val Loss: 0.063509, Val Acc: 98.33%

Epoch 55/100: Train Loss: 0.173356, Train Acc: 95.07%; Val Loss: 0.065841, Val Acc: 98.50%

Epoch 56/100: Train Loss: 0.155642, Train Acc: 95.52%; Val Loss: 0.114868, Val Acc: 96.67%

Epoch 57/100: Train Loss: 0.158389, Train Acc: 95.22%; Val Loss: 0.098776, Val Acc: 97.67%

Epoch 58/100: Train Loss: 0.151696, Train Acc: 95.24%; Val Loss: 0.101685, Val Acc: 97.33%

Epoch 59/100: Train Loss: 0.153997, Train Acc: 95.43%; Val Loss: 0.068799, Val Acc: 98.00%

Epoch 60/100: Train Loss: 0.153888, Train Acc: 95.09%; Val Loss: 0.084804, Val Acc: 97.50%

Epoch 61/100: Train Loss: 0.154668, Train Acc: 95.26%; Val Loss: 0.078420, Val Acc: 98.00%

Epoch 62/100: Train Loss: 0.148583, Train Acc: 95.37%; Val Loss: 0.074491, Val Acc: 97.83%

Epoch 63/100: Train Loss: 0.156984, Train Acc: 95.20%; Val Loss: 0.046452, Val Acc: 98.33%

Epoch 64/100: Train Loss: 0.156549, Train Acc: 95.33%; Val Loss: 0.091293, Val Acc: 97.83%

Epoch 65/100: Train Loss: 0.178375, Train Acc: 94.94%; Val Loss: 0.062937, Val Acc: 98.33%

Epoch 66/100: Train Loss: 0.164008, Train Acc: 95.20%; Val Loss: 0.083698, Val Acc: 97.00%

Epoch 67/100: Train Loss: 0.163003, Train Acc: 95.09%; Val Loss: 0.071845, Val Acc: 98.17%

Epoch 68/100: Train Loss: 0.145724, Train Acc: 95.37%; Val Loss: 0.081767, Val Acc: 98.33%

Epoch 69/100: Train Loss: 0.162443, Train Acc: 95.30%; Val Loss: 0.057950, Val Acc: 98.17%

Epoch 70/100: Train Loss: 0.150655, Train Acc: 95.70%; Val Loss: 0.127073, Val Acc: 96.50%

Epoch 71/100: Train Loss: 0.142713, Train Acc: 95.43%; Val Loss: 0.086860, Val Acc: 98.17%

Epoch 72/100: Train Loss: 0.132118, Train Acc: 95.89%; Val Loss: 0.085062, Val Acc: 98.00%

Epoch 73/100: Train Loss: 0.134143, Train Acc: 95.98%; Val Loss: 0.064904, Val Acc: 98.00%

Epoch 74/100: Train Loss: 0.151056, Train Acc: 95.78%; Val Loss: 0.067803, Val Acc: 98.17%

Epoch 75/100: Train Loss: 0.145741, Train Acc: 95.76%; Val Loss: 0.057462, Val Acc: 98.33%

Epoch 76/100: Train Loss: 0.138012, Train Acc: 95.81%; Val Loss: 0.068050, Val Acc: 98.50%

Epoch 77/100: Train Loss: 0.157364, Train Acc: 95.56%; Val Loss: 0.063588, Val Acc: 98.17%

Epoch 78/100: Train Loss: 0.136014, Train Acc: 95.74%; Val Loss: 0.107845, Val Acc: 97.83%

Epoch 79/100: Train Loss: 0.138868, Train Acc: 95.52%; Val Loss: 0.090419, Val Acc: 97.50%

Epoch 80/100: Train Loss: 0.141993, Train Acc: 95.85%; Val Loss: 0.068819, Val Acc: 98.17%

Epoch 81/100: Train Loss: 0.132294, Train Acc: 95.85%; Val Loss: 0.073918, Val Acc: 98.33%

Epoch 82/100: Train Loss: 0.139675, Train Acc: 95.85%; Val Loss: 0.060362, Val Acc: 98.33%

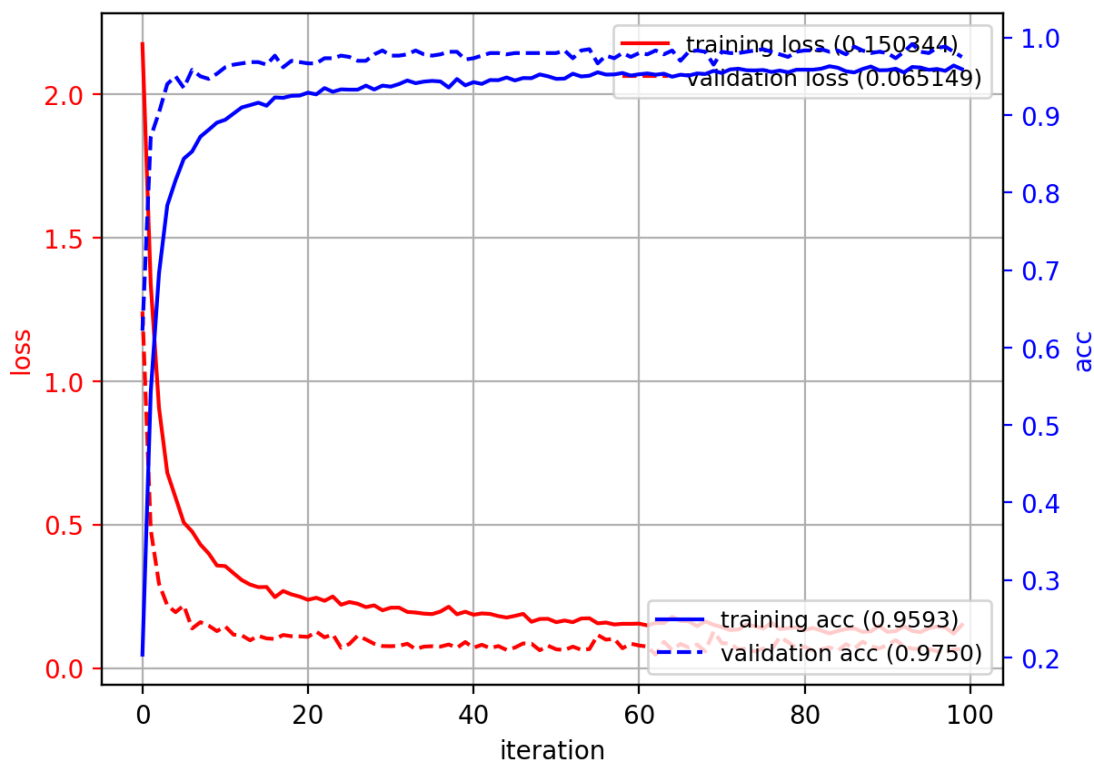
Epoch 83/100: Train Loss: 0.130533, Train Acc: 95.98%; Val Loss: 0.065063, Val Acc: 98.50%

Epoch 84/100: Train Loss: 0.120370, Train Acc: 96.30%; Val Loss: 0.071763, Val Acc: 98.17%

Epoch 85/100: Train Loss: 0.129061, Train Acc: 96.11%; Val Loss: 0.060187, Val Acc: 98.33%

Epoch 86/100: Train Loss: 0.135671, Train Acc: 95.65%; Val Loss: 0.082930, Val Acc: 97.83%

Epoch 87/100: Train Loss: 0.137978, Train Acc: 95.50%; Val Loss: 0.067844,
 Val Acc: 98.83%
 Epoch 88/100: Train Loss: 0.125886, Train Acc: 95.94%; Val Loss: 0.091414,
 Val Acc: 98.33%
 Epoch 89/100: Train Loss: 0.127593, Train Acc: 96.26%; Val Loss: 0.076080,
 Val Acc: 98.00%
 Epoch 90/100: Train Loss: 0.146653, Train Acc: 95.78%; Val Loss: 0.068365,
 Val Acc: 98.17%
 Epoch 91/100: Train Loss: 0.125724, Train Acc: 95.80%; Val Loss: 0.079791,
 Val Acc: 98.00%
 Epoch 92/100: Train Loss: 0.128655, Train Acc: 95.98%; Val Loss: 0.097038,
 Val Acc: 97.33%
 Epoch 93/100: Train Loss: 0.145341, Train Acc: 95.50%; Val Loss: 0.076184,
 Val Acc: 98.17%
 Epoch 94/100: Train Loss: 0.129089, Train Acc: 96.20%; Val Loss: 0.067117,
 Val Acc: 99.17%
 Epoch 95/100: Train Loss: 0.125057, Train Acc: 96.13%; Val Loss: 0.065621,
 Val Acc: 98.17%
 Epoch 96/100: Train Loss: 0.138797, Train Acc: 95.83%; Val Loss: 0.065958,
 Val Acc: 98.00%
 Epoch 97/100: Train Loss: 0.144240, Train Acc: 95.94%; Val Loss: 0.058420,
 Val Acc: 98.67%
 Epoch 98/100: Train Loss: 0.141444, Train Acc: 95.70%; Val Loss: 0.052311,
 Val Acc: 98.83%
 Epoch 99/100: Train Loss: 0.119762, Train Acc: 96.39%; Val Loss: 0.064954,
 Val Acc: 98.17%
 Epoch 100/100: Train Loss: 0.150344, Train Acc: 95.93%; Val Loss: 0.065149,
 Val Acc: 97.50%



17 Tensorboard

- Tensorboard runs as a local webserver to view the results in the log directory.
 - to start Tensorboard, run the following command in the shell: `tensorboard --logdir ./logs`
 - then connect to the local webserver using a browser (the URL is printed out when Tensorboard starts).
- data augmentation increases accuracy.

```
[29]: acc = evaluate_model(model, test_loader)
      print("test accuracy:", acc)
```

test accuracy: 98.57

18 Data augmentation with noise

- Also add per-pixel noise to the image for data augmentation.
 - define a function to add noise
 - set it as the `preprocessing_function`

```
[30]: transform_noise = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomRotation(degrees=10),           # rotation_range=10
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)), # width_shift_range=0.1, height_shift_range=0.1
    transforms.RandomAffine(degrees=0, shear=10),     # shear_range=0.1
    transforms.RandomResizedCrop(28, scale=(0.9, 1.1)), # zoom_range=0.1
    transforms.ToTensor(),
    transforms.Lambda(lambda x: torch.clamp(x + torch.randn_like(x) * np.sqrt(0.05), 0, 1)),
])

train_noise_dataset = MyDataset(vtrainI,vtrainY,transform_noise)
```

- Example: original image (top-left) and augmented data

```
[31]: imgs = []

cnt = 0
for augx, y in train_noise_dataset:
    imgs.append(augx[0].numpy())
    cnt += 1
    if cnt > 8:
        # we need to break the loop by hand because
        # the generator loops indefinitely
        break
```

```
[32]: plt.figure(figsize=(8,4))
show_imgs(imgs, nc=5)
plt.show()
```



- Train with augmented data: transformations and per-pixel noise

```
[33]: model = CNNWithDropout()

# Print model architecture
print(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

train_noise_loader = DataLoader(train_noise_dataset, batch_size=batch_size,
    ↪shuffle=True)

optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_noise_loader, valid_loader,
    ↪criterion, optimizer, epochs=100, patience=-1, writer='runs/daug-noise')

# Plot the training history
plot_history(history)
```

```
CNNWithDropout(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (softmax): Softmax(dim=1)
  (drop1d): Dropout(p=0.5, inplace=False)
  (drop2d): Dropout2d(p=0.5, inplace=False)
)
Using device: cuda
Epoch 1/100:   Train Loss: 2.291393, Train Acc: 12.98%;   Val Loss: 2.155801,
Val Acc: 35.50%
Epoch 2/100:   Train Loss: 1.689179, Train Acc: 42.02%;   Val Loss: 0.836176,
Val Acc: 76.00%
Epoch 3/100:   Train Loss: 1.131734, Train Acc: 61.33%;   Val Loss: 0.421438,
Val Acc: 87.50%
Epoch 4/100:   Train Loss: 0.841336, Train Acc: 72.65%;   Val Loss: 0.319056,
Val Acc: 90.67%
Epoch 5/100:   Train Loss: 0.677139, Train Acc: 78.31%;   Val Loss: 0.276410,
Val Acc: 92.17%
```

Epoch 6/100: Train Loss: 0.597790, Train Acc: 80.91%; Val Loss: 0.216478, Val Acc: 93.83%

Epoch 7/100: Train Loss: 0.518135, Train Acc: 84.50%; Val Loss: 0.228166, Val Acc: 94.00%

Epoch 8/100: Train Loss: 0.486356, Train Acc: 85.04%; Val Loss: 0.148791, Val Acc: 95.50%

Epoch 9/100: Train Loss: 0.454146, Train Acc: 85.72%; Val Loss: 0.188656, Val Acc: 94.00%

Epoch 10/100: Train Loss: 0.445433, Train Acc: 86.54%; Val Loss: 0.135028, Val Acc: 96.17%

Epoch 11/100: Train Loss: 0.406133, Train Acc: 87.76%; Val Loss: 0.106894, Val Acc: 96.67%

Epoch 12/100: Train Loss: 0.393248, Train Acc: 87.85%; Val Loss: 0.125601, Val Acc: 95.50%

Epoch 39/100: Train Loss: 0.239046, Train Acc: 92.65%; Val Loss: 0.095709, Val Acc: 98.00%

Epoch 40/100: Train Loss: 0.237396, Train Acc: 93.17%; Val Loss: 0.097174, Val Acc: 97.17%

Epoch 41/100: Train Loss: 0.208919, Train Acc: 93.91%; Val Loss: 0.090380, Val Acc: 97.50%

Epoch 42/100: Train Loss: 0.221190, Train Acc: 93.78%; Val Loss: 0.094726, Val Acc: 97.33%

Epoch 43/100: Train Loss: 0.234106, Train Acc: 92.89%; Val Loss: 0.075665, Val Acc: 97.50%

Epoch 44/100: Train Loss: 0.211534, Train Acc: 93.43%; Val Loss: 0.077956, Val Acc: 97.50%

Epoch 45/100: Train Loss: 0.197883, Train Acc: 94.09%; Val Loss: 0.080788, Val Acc: 97.83%

Epoch 46/100: Train Loss: 0.213383, Train Acc: 93.65%; Val Loss: 0.080123, Val Acc: 98.17%

Epoch 47/100: Train Loss: 0.217864, Train Acc: 93.11%; Val Loss: 0.109646, Val Acc: 97.83%

Epoch 48/100: Train Loss: 0.203814, Train Acc: 93.72%; Val Loss: 0.070675, Val Acc: 98.17%

Epoch 49/100: Train Loss: 0.233557, Train Acc: 92.91%; Val Loss: 0.069544, Val Acc: 97.83%

Epoch 50/100: Train Loss: 0.191292, Train Acc: 94.00%; Val Loss: 0.065648, Val Acc: 98.17%

Epoch 51/100: Train Loss: 0.199351, Train Acc: 94.07%; Val Loss: 0.082896, Val Acc: 97.83%

Epoch 52/100: Train Loss: 0.200981, Train Acc: 93.74%; Val Loss: 0.091617, Val Acc: 98.00%

Epoch 53/100: Train Loss: 0.203324, Train Acc: 93.91%; Val Loss: 0.055488, Val Acc: 98.67%

Epoch 54/100: Train Loss: 0.200608, Train Acc: 94.13%; Val Loss: 0.095012, Val Acc: 97.67%

Epoch 55/100: Train Loss: 0.217101, Train Acc: 93.80%; Val Loss: 0.099519, Val Acc: 96.83%

Epoch 56/100: Train Loss: 0.194027, Train Acc: 94.35%; Val Loss: 0.089238, Val Acc: 97.67%

Epoch 57/100: Train Loss: 0.198733, Train Acc: 94.07%; Val Loss: 0.075073, Val Acc: 98.00%

Epoch 58/100: Train Loss: 0.198838, Train Acc: 94.43%; Val Loss: 0.072531, Val Acc: 98.33%

Epoch 59/100: Train Loss: 0.195004, Train Acc: 93.85%; Val Loss: 0.074526, Val Acc: 98.00%

Epoch 60/100: Train Loss: 0.191691, Train Acc: 94.52%; Val Loss: 0.081587, Val Acc: 98.17%

Epoch 61/100: Train Loss: 0.196884, Train Acc: 94.39%; Val Loss: 0.067878, Val Acc: 98.00%

Epoch 62/100: Train Loss: 0.190495, Train Acc: 94.31%; Val Loss: 0.069026, Val Acc: 98.33%

Epoch 63/100: Train Loss: 0.182916, Train Acc: 94.56%; Val Loss: 0.072900, Val Acc: 97.83%

Epoch 64/100: Train Loss: 0.181699, Train Acc: 94.65%; Val Loss: 0.056620, Val Acc: 98.17%

Epoch 65/100: Train Loss: 0.186544, Train Acc: 94.70%; Val Loss: 0.081650, Val Acc: 98.33%

Epoch 66/100: Train Loss: 0.178221, Train Acc: 94.52%; Val Loss: 0.070199, Val Acc: 98.33%

Epoch 67/100: Train Loss: 0.174976, Train Acc: 95.15%; Val Loss: 0.064783, Val Acc: 97.50%

Epoch 68/100: Train Loss: 0.185915, Train Acc: 94.52%; Val Loss: 0.074554, Val Acc: 98.00%

Epoch 69/100: Train Loss: 0.176306, Train Acc: 95.04%; Val Loss: 0.084118, Val Acc: 97.83%

Epoch 70/100: Train Loss: 0.189226, Train Acc: 94.43%; Val Loss: 0.081173, Val Acc: 98.00%

Epoch 71/100: Train Loss: 0.191017, Train Acc: 94.50%; Val Loss: 0.046923, Val Acc: 98.33%

Epoch 72/100: Train Loss: 0.180539, Train Acc: 94.61%; Val Loss: 0.069087, Val Acc: 97.83%

Epoch 73/100: Train Loss: 0.177629, Train Acc: 94.52%; Val Loss: 0.080316, Val Acc: 97.67%

Epoch 74/100: Train Loss: 0.151999, Train Acc: 95.30%; Val Loss: 0.093799, Val Acc: 98.00%

Epoch 75/100: Train Loss: 0.182437, Train Acc: 94.39%; Val Loss: 0.100832, Val Acc: 97.33%

Epoch 76/100: Train Loss: 0.168903, Train Acc: 94.69%; Val Loss: 0.091121, Val Acc: 97.67%

Epoch 77/100: Train Loss: 0.172601, Train Acc: 95.06%; Val Loss: 0.113424, Val Acc: 97.33%

Epoch 78/100: Train Loss: 0.175012, Train Acc: 94.50%; Val Loss: 0.071240, Val Acc: 98.17%

Epoch 79/100: Train Loss: 0.174367, Train Acc: 95.04%; Val Loss: 0.085942, Val Acc: 98.00%

Epoch 80/100: Train Loss: 0.167677, Train Acc: 95.09%; Val Loss: 0.049685, Val Acc: 99.17%

Epoch 81/100: Train Loss: 0.156449, Train Acc: 95.22%; Val Loss: 0.062329, Val Acc: 99.00%

Epoch 82/100: Train Loss: 0.187949, Train Acc: 94.52%; Val Loss: 0.077483, Val Acc: 98.00%

Epoch 83/100: Train Loss: 0.175390, Train Acc: 95.04%; Val Loss: 0.088621, Val Acc: 97.83%

Epoch 84/100: Train Loss: 0.163592, Train Acc: 95.11%; Val Loss: 0.116471, Val Acc: 97.67%

Epoch 85/100: Train Loss: 0.168314, Train Acc: 95.00%; Val Loss: 0.069394, Val Acc: 98.67%

Epoch 86/100: Train Loss: 0.169363, Train Acc: 95.02%; Val Loss: 0.083280, Val Acc: 97.67%

Epoch 87/100: Train Loss: 0.178365, Train Acc: 95.02%; Val Loss: 0.054288, Val Acc: 98.83%

Epoch 88/100: Train Loss: 0.181609, Train Acc: 94.69%; Val Loss: 0.078866, Val Acc: 98.33%

Epoch 89/100: Train Loss: 0.156687, Train Acc: 95.56%; Val Loss: 0.061111, Val Acc: 98.83%

Epoch 90/100: Train Loss: 0.168110, Train Acc: 94.63%; Val Loss: 0.053329, Val Acc: 98.33%

Epoch 91/100: Train Loss: 0.180285, Train Acc: 94.78%; Val Loss: 0.079058, Val Acc: 98.00%

Epoch 92/100: Train Loss: 0.163414, Train Acc: 95.33%; Val Loss: 0.071954, Val Acc: 98.17%

Epoch 93/100: Train Loss: 0.177670, Train Acc: 95.06%; Val Loss: 0.068405, Val Acc: 98.17%

Epoch 94/100: Train Loss: 0.157426, Train Acc: 94.89%; Val Loss: 0.056947, Val Acc: 99.00%

Epoch 95/100: Train Loss: 0.175639, Train Acc: 94.81%; Val Loss: 0.063903, Val Acc: 98.17%

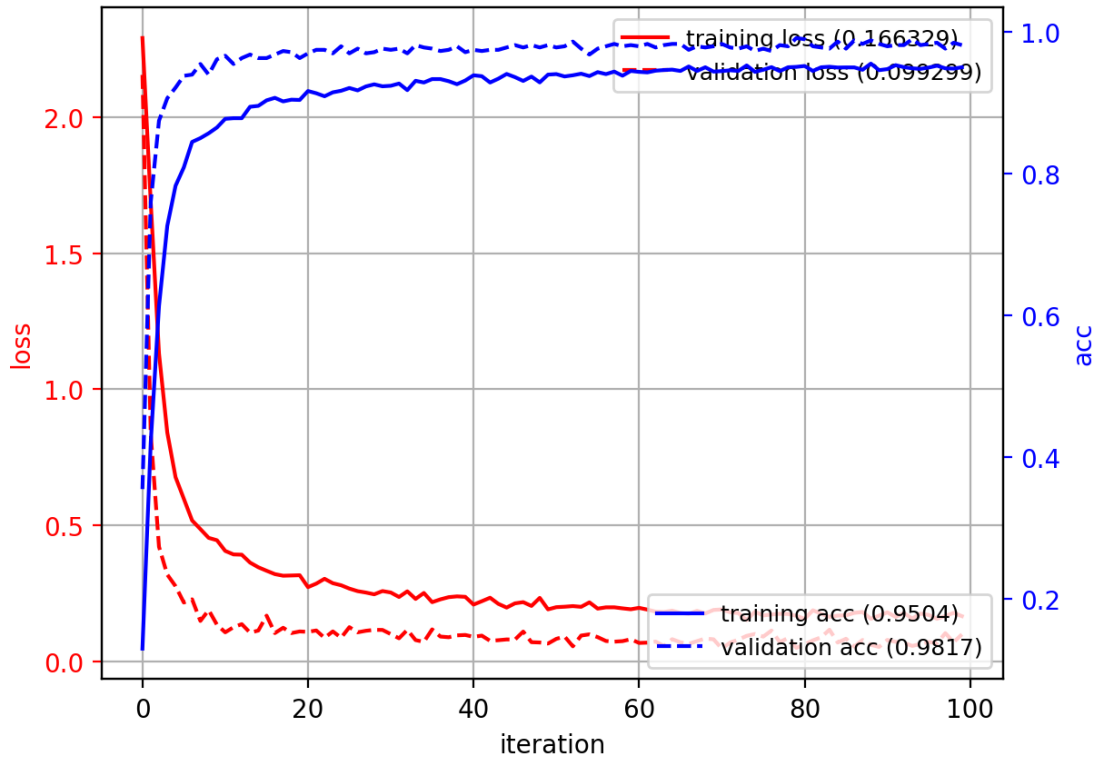
Epoch 96/100: Train Loss: 0.170718, Train Acc: 95.00%; Val Loss: 0.071549, Val Acc: 98.33%

Epoch 97/100: Train Loss: 0.156655, Train Acc: 95.26%; Val Loss: 0.066612, Val Acc: 98.50%

Epoch 98/100: Train Loss: 0.166869, Train Acc: 94.89%; Val Loss: 0.103912, Val Acc: 97.67%

Epoch 99/100: Train Loss: 0.178384, Train Acc: 94.85%; Val Loss: 0.070454, Val Acc: 98.50%

Epoch 100/100: Train Loss: 0.166329, Train Acc: 95.04%; Val Loss: 0.099299, Val Acc: 98.17%



- Similar results.

```
[34]: acc = evaluate_model(model, test_loader)
      print("test accuracy:", acc)
```

test accuracy: 98.75

19 Data generator options

- Sometimes the dataset is too large to store completely in RAM:
 - keep the path as `text` and iterate over paths to read images in a directory

20 Comparison on MNIST

- For image data, CNN works better and has less parameters.

Type

No.Layers

Architecture

No.Parameters

Test Accuracy

LR

1

output(10)
 7,850
 0.8911
 MLP
 2
 ReLu(50), output(10)
 39,760
 0.9380
 MLP
 2
 Relu(200), output(10)
 159,010
 0.9437
 MLP
 2
 Relu(1000), output(10)
 795,010
 0.9462
 MLP
 3
 ReLu(500), Relu(500), output(10)
 648,010
 0.9529
 CNN
 3
 Conv(10x5x5), ReLu(50), output(10)
 98,820
 0.9534
 CNN
 5
 Conv(10x5x5), Conv(50x3x3), Conv(80x3x3), ReLu(50), output(10)
 286,940
 0.9564
 CNN (w/ weight decay)
 5
 Conv(10x5x5), Conv(50x3x3), Conv(80x3x3), ReLu(50), output(10)
 286,940
 0.9660
 CNN (ensemble)
 5
 Conv(10x5x5), Conv(50x3x3), Conv(80x3x3), ReLu(50), output(10)
 5x286,940
 0.9723
 CNN (w/ dropout)
 5
 Conv(10x5x5), Conv(50x3x3), Conv(80x3x3), ReLu(50), output(10)
 286,940
 0.9713

CNN (w/ dropout, data-augmentation)

5

Conv(10x5x5), Conv(50x3x3), Conv(80x3x3), ReLu(50), output(10)

286,940

0.9893

CNN (w/ dropout, data-augmentation, noise)

5

Conv(10x5x5), Conv(50x3x3), Conv(80x3x3), ReLu(50), output(10)

286,940

0.9895

21 Summary

- **Convolutional neural network (CNN)**
 - convolution filters for extracting local image features
 - * global translation equivariance
 - local translation invariance using max-pooling
 - classifier using MLP on the image features.
- **Regularization methods**
 - L2-norm regularization (weight-decay) - keep weights from becoming too large
 - Early-stopping - adaptively keep weights from becoming too large
 - Ensembles - reduce error by bagging
 - Dropout - approximate an ensemble of networks
 - Data Augmentation - make the NN robust to various image transformations